

Catching Every Ripple: Enhanced Anomaly Awareness via Dynamic Concept Adaptation

Jiaqi Zhu, Shaofeng Cai, Jie Chen, *Fellow, IEEE*, Fang Deng, *Fellow, IEEE*,
Beng Chin Ooi, *Fellow, IEEE*, and Wenqiao Zhang

Abstract—Online anomaly detection (OAD) plays a pivotal role in real-time analytics and decision-making for evolving data streams. However, existing methods often rely on costly retraining and rigid decision boundaries, limiting their ability to adapt both effectively and efficiently to concept drift in dynamic environments. To address these challenges, we propose DyMETER, a dynamic concept adaptation framework for OAD that unifies on-the-fly parameter shifting and dynamic thresholding within a single online paradigm. DyMETER first learns a static detector on historical data to capture recurring *central concepts*, and then transitions to a dynamic mode to adapt to *new concepts* as drift occurs. Specifically, DyMETER employs a novel dynamic concept adaptation mechanism that leverages a hypernetwork to generate instance-aware parameter shifts for the static detector, thereby enabling efficient and effective adaptation without retraining or fine-tuning. To achieve robust and interpretable adaptation, DyMETER introduces a lightweight evolution controller to estimate instance-level concept uncertainty for adaptive updates. Further, DyMETER employs a dynamic threshold optimization module to adaptively recalibrate the decision boundary by maintaining a candidate window of uncertain samples, which ensures continuous alignment with evolving concepts. Extensive experiments demonstrate that DyMETER significantly outperforms existing OAD approaches across a wide spectrum of application scenarios.

Index Terms—Online Anomaly Detection, Concept Drift, Streaming Data, Dynamic Thresholding.

I. INTRODUCTION

ANOMALY detection (AD) plays a pivotal role across various domains by uncovering hidden irregularities that deepen data understanding, enable timely interventions, and support reliable decision-making in complex, data-driven systems [1], [2], [3], [4], [5]. While extensive progress has been made in detecting anomalies in static data settings [6], [7], [8],

[9], extending AD to streaming environments remains substantially more challenging. This task, known as *online anomaly detection (OAD)*, aims to identify anomalous instances in real time from evolving data streams to preserve data integrity and ensure operational reliability across critical applications.

In OAD tasks, *concept drift* poses the central challenge, as unpredictable shifts in the underlying data distribution or correlation structure undermine the validity of models trained on historical data over time. While traditional static AD approaches may perform well within specific periods, they quickly lose efficacy once concept drift occurs, failing to capture newly emerging abnormal behaviors. In this context, an anomaly is defined relative to the current nominal distribution of the data stream, which evolves over time. A pertinent example arises in the financial domain, where transaction patterns evolve due to market trends, seasonal fluctuations, or new trading strategies. This requires that fraudulent activities be identified relative to the nominal distribution. Since anomalies are inherently context-dependent, a score that indicates normality under one concept may correspond to an anomaly under another, rendering static thresholds ineffective.

Unlike concept drift detection, which focuses on identifying when distributional changes occur, OAD under concept drift operates at a finer granularity by continuously discerning rare instance-level deviations within the evolving nominal distribution. Notably, a concept change point itself is not necessarily anomalous. For instance, a shift from daily to holiday transactions in financial monitoring redefines the nominal concept, whereas anomalies are deviations within this new concept, such as fraudulent transactions concealed among legitimate holiday purchases. This distinction highlights OAD's unique challenges: beyond merely signaling drift, it requires maintaining sensitivity to subtle, context-dependent anomalies amid continuously evolving concepts.

Recently, several preliminary attempts have been proposed to support OAD in evolving data streams, as depicted in Figure 1. *Incremental methods* [10], [11], [12], [13], [14] typically construct an initial model that is gradually updated as new data arrives. However, they often rely on repeated retraining or fine-tuning, resulting in high computational cost and slow responsiveness to emerging concepts. *Ensemble methods* [8], [15], [16] maintain multiple models tailored to different concepts, but their effectiveness is constrained by the size and diversity of the model pool, while sustaining and updating multiple models incurs substantial computational and memory overhead. More recently, *drift-adaptive architectures* [17], [18], [19], [20] embed adaptation mechanisms directly into model

Manuscript received March 22, 2025. This work was supported in part by the National Natural Science Foundation of China National Science Fund for Young Scientists (Ph.D.) under Grant 624B2027, in part by the National Science and Technology Major Project under Grant 2022ZD0119701, in part by the National Natural Science Foundation of China National Science Fund for Distinguished Young Scholars under Grant 62025301.

Jiaqi Zhu and Fang Deng are with the School of Automation and the National Key Laboratory of Autonomous Intelligent Unmanned Systems, Beijing Institute of Technology, Beijing 100081, China (e-mail: jiaqi_zhu@bit.edu.cn; dengfang@bit.edu.cn).

Shaofeng Cai and Beng Chin Ooi are with the School of Computing, National University of Singapore, Singapore 117417 (e-mail: shaofeng@comp.nus.edu.sg; ooi@comp.nus.edu.sg).

Jie Chen is with Harbin Institute of Technology, Harbin 150001, China, and also with the Beijing Institute of Technology, Beijing 100081, China (e-mail: chenjie@bit.edu.cn).

Wenqiao Zhang is with Digital Media Computing & Design Lab, Zhejiang University, Hangzhou 310000, China (e-mail: wenqiaozhang@zju.edu.cn).

Corresponding author: Fang Deng (dengfang@bit.edu.cn)

design. Nevertheless, they are typically tailored to specific drift patterns and thus lack flexibility across diverse drift scenarios. Despite effectiveness in specific settings, existing approaches share several fundamental challenges: (i) adaptation is typically driven by coarse window or batch-level signals rather than instance-level control, (ii) model adaptation often requires continuous tuning or maintaining multiple models instead of efficient inference-time adaptation, and (iii) decision boundaries are typically fixed, making them insensitive to subtle context-dependent anomalies under concept drift.

To address these challenges, we propose DyMETER, a novel online anomaly detection framework that reframes adaptation under concept drift as instance-aware, inference-time evolution with dynamic decision boundary calibration. The core insight behind DyMETER is that historical data often captures the dominant concept patterns and thus comprises the recurring *central concepts*, while newly arriving data typically deviate from these patterns in a gradual and localized manner. Guided by this understanding, instead of incremental updating, ensemble modeling, and drift-specific heuristics, DyMETER anchors detection to these central concepts and adapts to *emerging concepts* on the fly by jointly evolving the detector's behavior and its decision boundary during online detection as data streams evolve.

Specifically, DyMETER operationalizes dynamic concept adaptation by directly addressing the fundamental challenges of existing approaches. First, to enable principled instance-level control, DyMETER models concept uncertainty derived from the evidential deep learning (EDL) theory [21] for each incoming instance, providing a continuous and interpretable signal to determine when adaptation is necessary. Second, to achieve efficient concept adaptation, DyMETER performs inference-time model evolution by dynamically generating instance-conditioned parameter shifts via a hypernetwork. This allows the detector to rapidly re-center toward emerging concepts in a fine-grained and responsive manner, without inefficient gradient-based optimization. Finally, to maintain reliable anomaly decisions under evolving concepts, DyMETER incorporates cost-sensitive adaptive thresholding that jointly accounts for recent score statistics and concept uncertainty, enabling continuous decision boundary calibration. As a result, DyMETER attains a unified capability for instance-aware model evolution and dynamic decision calibration, effectively mitigating the coarse adaptation control, high adaptation cost, and decision miscalibration that commonly limit existing OAD methods under concept drift.

We summarize our main contributions as follows:

- We present DyMETER, a novel OAD framework that unifies inference-time adaptation and dynamic decision calibration to robustly handle concept drift in evolving data streams.
- We incorporate a lightweight evolution controller into DyMETER that enables evidential concept uncertainty modeling, supporting fine-grained adaptation control and interpretable anomaly detection.
- We propose a dynamic concept adaptation mechanism that efficiently re-centers the model toward evolving concept by generating instance-aware parameter shifts via a hypernetwork, avoiding continuous retraining or finetuning.

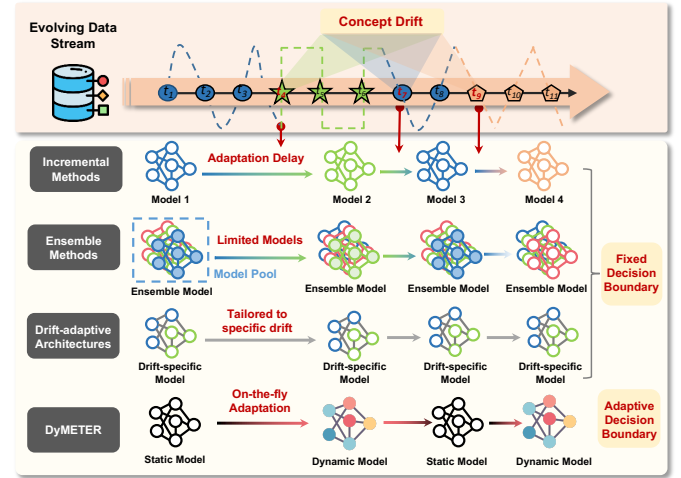


Fig. 1: Adaptation paradigms of DyMETER and existing approaches for handling concept drift in evolving data streams.

- Extensive experiments demonstrate that DyMETER consistently outperforms state-of-the-art OAD baselines across diverse drift scenarios.

The remainder of this paper is organized as follows: Section II introduces the preliminaries, including key concepts and background information. Section III presents our DyMETER framework, providing a comprehensive overview of its modules, optimization schemes, and discussions on its key characteristics. In Section IV, we showcase experimental results on various benchmarks. Section V reviews related work in the field. Section VI outlines limitations and future directions, and finally, Section VII concludes the paper.

II. PRELIMINARIES

A. Problem Formulation

Anomaly detection aims to identify data instances that diverge from established norms. Specifically, this paper focuses on unsupervised *online anomaly detection* (OAD), which seeks to identify anomalies within live data streams without labeled information, as delineated in Definition 1. The task becomes particularly challenging under *concept drift*, a phenomenon where the underlying statistical or distributional attributes of a dataset evolve over time, as described in Definition 2. Such dynamics make it difficult to maintain reliable anomaly detection in real-world scenarios.

Definition 1 (Online Anomaly Detection). *Given an incoming data stream $\mathcal{X} = \{\vec{x}_1, \dots, \vec{x}_t, \dots\}$, where each instance $\vec{x}_t = (x_{t1}, \dots, x_{td})$ is a multivariate feature vector, online anomaly detection aims to determine whether $\vec{x}_t$ is anomalous or not at each time step t .*

Definition 2 (Concept Drift). *Concept drift occurs at time step t when the underlying joint probability $P(\vec{x}, y)$ of input data $\vec{x}$ and its corresponding label y changes at time t , such that, $P(\vec{x}_t, y_t) \neq P(\vec{x}, y)$.*

In practice, the input instance $\vec{x}_t$ comprises d attributes that can be either categorical or numerical depending on the application domain. For example, in financial transaction

monitoring, $\vec{x}_t$ characterizes a transaction at time t with dimensions such as amount, merchant type, and geolocation. As these attributes evolve, concept drift can cause the same anomaly score to indicate normal behavior under one concept but an anomaly under another, such as high-amount transactions being typical during holiday seasons yet anomalous in routine periods. Therefore, it is crucial for the model to grasp the evolving dynamics of the data stream and adapt to concept drifts effectively over time. This involves dynamically generating an anomaly score from a detection model for each data instance $\vec{x}_t$, assessing its deviation from normal behavior within the stream's varying concepts. A sample is then identified as anomalous if its anomaly score surpasses an adaptive threshold, which continuously refines the detection criteria to remain aligned with evolving data characteristics. Table I summarizes the key notations used throughout the paper to facilitate quick reference and improve readability.

B. Hypernetwork

Hypernetworks function as a meta neural network that dynamically produces the parameters for a secondary *primary network*, enabling it to adjust its parameters based on specific inputs. In our framework, a hypernetwork modifies the parameters of our base model, serving as the primary network to respond to evolving concepts in the data stream. Specifically, we evaluate the parameter changes within our base model, which consists of N_l layers. For each layer, the hypernetwork assigns a parameter matrix $K^{(n)} \in \mathbb{R}^{N_{in} \times N_{out}}$, where N_{in} and N_{out} denote the input and output dimensionalities of the layer, respectively. This process is conceptualized as follows:

$$K^{(n)} = \xi(\vec{r}^{(n)}; \Theta_h), \forall n = 1, \dots, N_l \quad (1)$$

where $\vec{r}^{(n)}$ is a random vector, and $\xi(\cdot)$ denotes a randomly initialized multi-layer perceptron (MLP) parameterized by Θ_h . This setup allows for backpropagation to $\vec{r}^{(n)}$ and $\xi(\cdot)$, ensuring the primary network adapts effectively to the data stream through end-to-end training.

C. Evidential Deep Learning

Evidential Deep Learning (EDL) [21] advances a probabilistic framework within deep learning by treating neural network outputs as distributions over potential class probabilities, specifically by applying a Dirichlet prior to these probabilities. This methodology extends beyond providing mere point predictions by offering uncertainty estimates for each output. Concretely, in typical C-class classification tasks, a deep neural network (DNN) uses a softmax function to transform instance $\vec{x}$ outputs into class probability vectors $\vec{p}$. EDL enhances this by applying a Dirichlet distribution over $\vec{p}$, effectively modeling each class's probability distribution for $\vec{x}$. The Dirichlet distribution, formulated as:

$$P(\vec{p}|\vec{x}; \Theta_e) = Dir(\vec{p}|\vec{\alpha}) = \begin{cases} \frac{\Gamma(\sum_{c=1}^C \alpha_c)}{\prod_{c=1}^C \Gamma(\alpha_c)} \prod_{c=1}^C p_c^{\alpha_c-1}, & \text{if } \vec{p} \in \Delta^C \\ 0, & \text{otherwise} \end{cases} \quad (2)$$

where Θ_e denotes the parameters of the evidential classifier that maps an input $\vec{x}$ to Dirichlet evidence. The Dirichlet

TABLE I: Summary of key notations.

Notation	Description
$\mathcal{X} = \{\vec{x}_1, \dots, \vec{x}_t, \dots\}$	Data stream, where $\vec{x}_t$ denotes the input instance at time t .
$\Theta_s = (\Theta_s^E, \Theta_s^D)$	Parameters of the SCD.
$\Theta_d = (\Theta_d^E, \Theta_d^D)$	Parameters of the DSD.
$\Delta\Theta_d = (\Delta\Theta_d^E, \Delta\Theta_d^D)$	Parameter shifts produced by the hypernetwork.
$\mathcal{E}_s(\cdot), \mathcal{D}_s(\cdot)$	Encoder and decoder of SCD.
$\mathcal{E}_d(\cdot), \mathcal{D}_d(\cdot)$	Encoder and decoder of DSD.
$\vec{y}, \vec{y}'$	Reconstructed outputs of SCD and DSD.
$\vec{z}, \vec{z}'$	Latent representation vectors of SCD and DSD.
Θ_c	Parameters of the IEC.
$\tilde{l}$	Pseudo label for IEC training.
$\vec{\alpha}, \alpha_c$	Dirichlet concentration vector and its component for class c .
$\mathcal{U}_{ctr}$	Concept uncertainty estimated by the IEC.
μ_e	Uncertainty threshold used to filter unreliable samples.
μ_p	Reconstruction error threshold for pseudo-labeling.
$\mathcal{A}(\cdot)$	Anomaly score assigned to an input sample.
λ	Weight that scales the effect of uncertainty on anomaly score.
$\mathcal{F}_c(\cdot)$	Expected cost function used for dynamic thresholding.
$\mathcal{W}_N, \mathcal{W}_C$	Sliding windows for normal and uncertain samples.
r_e	Reference reconstruction error for anomaly score scaling.
μ_a^*	Final adaptive decision threshold for anomaly detection.
τ, κ	Hyperparameters for quantile level and regularization.
μ_o	Threshold for triggering offline update.

distribution is parameterized by $\vec{\alpha}$, which are predicted by the evidential classifier via $\vec{\alpha} = g(f(\vec{x}))$, where $f(\cdot)$ denotes the classifier's output mapping from $\vec{x}$ to evidence, and $g(\cdot)$ is an exponential function enforcing non-negativity. $\Gamma(\cdot)$ represents the Gamma function, and Δ^C is the C -dimensional unit simplex, $\Delta^C = \{\sum_{c=1}^C p_c = 1 \text{ and } 0 \leq p_c \leq 1\}$. This probabilistic approach allows predictions for $\vec{x}$ to reflect a range of possible outcomes, enriching the model's interpretability and capability to handle uncertainty, particularly valuable for managing concept drift in dynamic environments. Within the DyMETER framework, EDL plays a vital role in quantifying concept uncertainty, facilitating the model evolution based on the evaluated uncertainty of each input.

III. METHODOLOGY

A. Overview of DyMETER

DyMETER is a dynamic concept adaptation framework for online anomaly detection under evolving concepts, enabling instance-aware adaptation of both detector parameters and decision boundaries at inference time. At a high level, DyMETER comprises four tightly coupled components. The Static Concept-aware Detector (SCD) learns central concepts from historical data and provides anomaly evidence for the Intelligent Evolution Controller (IEC), which estimates concept uncertainty to determine whether an incoming instance aligns with established concepts or signals potential evolution. Guided by this uncertainty, the Dynamic Shift-aware Detector (DSD) selectively adapts detector parameters at the instance level, while the Dynamic Threshold Optimization (DTO) continuously calibrates the anomaly decision boundary to ensure robustness under evolving data streams. Through this coordinated design, DyMETER forms a unified uncertainty-guided adaptation framework for robust and responsive online anomaly detection under evolving concept drifts.

B. Static Concept-aware Detector

The Static Concept-aware Detector (SCD) is designed to identify anomalies within the central concepts of a data stream

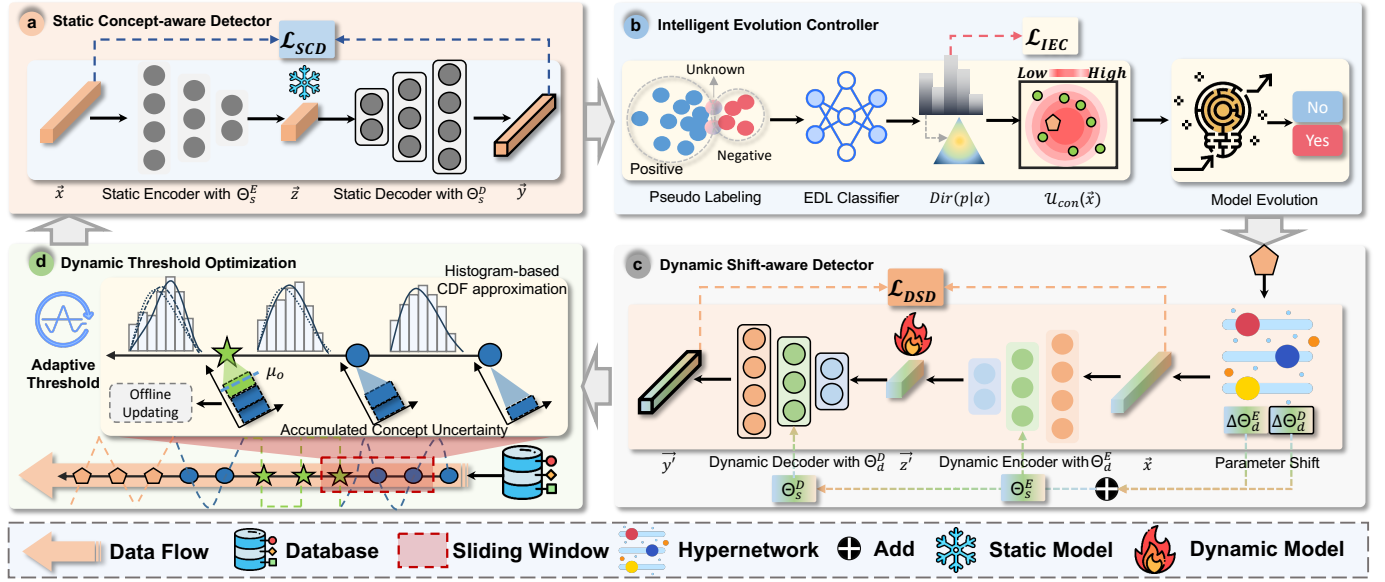


Fig. 2: Overview of the proposed DyMETER. (a) Static Concept-aware Detector (SCD) is trained on historical data to model the central concepts. (b) Intelligent Evolution Controller (IEC) timely measures the concept uncertainty to determine the necessity of model evolution. (c) Dynamic Shift-aware Detector (DSD) dynamically updates SCD with the instance-aware parameter shift by considering the concept drift. (d) Dynamic Threshold Optimization (DTO) establishes a cost-sensitive decision boundary by tracking the distribution of reconstruction errors, followed by an accumulated concept uncertainty-based updating strategy.

by capturing the overall distribution of historical data. The SCD uses an autoencoder, known for its effectiveness and unsupervised nature [22], [23], as the static detector to effectively identify predominant anomaly types within the central concepts of unlabeled data streams.

Static Detector. The workflow of our static detector operates in two primary phases: encoding and decoding. Concretely, given an input instance $\vec{x}$ in the data stream $\mathcal{X}$, the encoder and decoder jointly form an autoencoder-based detector with parameters $\Theta_s = (\Theta_s^E, \Theta_s^D)$. The encoder $\mathcal{E}_s(\cdot; \Theta_s^E)$ compresses the representation of the input $\vec{x}$ into a latent vector $\vec{z}$, and subsequently the decoder $\mathcal{D}_s(\cdot; \Theta_s^D)$ reconstructs the original input using the latent vector $\vec{z}$ as follows:

$$\vec{x} \Rightarrow \mathcal{E}_s(\vec{x}; \Theta_s^E) = \vec{z} \Rightarrow \mathcal{D}_s(\vec{z}; \Theta_s^D) = \vec{y} \quad (3)$$

The static detector is initially trained on a small subset of the historical data stream $\mathcal{X}_h$, which calculates the reconstruction error by measuring the squared deviations between each input and its reconstructed output, denoted by $L_2(\cdot)$, formulated as:

$$\mathcal{L}_{SCD}(\Theta_s) = L_2(\vec{x}, \vec{y}) = \frac{\sum_{i=1}^n (\vec{x}_i - \vec{y}_i)^2}{n}, \quad (4)$$

where n denotes the dimension of each input instance. By constraining the model to internalize essential relationships and features of the input instances via $\mathcal{L}_{SCD}$, the autoencoder-based static detector functions effectively as an identity mapper for its training distribution. This setup allows it to reliably regenerate unseen samples from $\mathcal{X}_h$, where anomalies are revealed through elevated reconstruction errors.

C. Intelligent Evolution Controller

The reconstruction error from the static detector serves as an indicator of normality or anomaly in the input data.

However, its reliability decreases with out-of-distribution data, particularly under conditions of concept drift [24], [25]. To address this challenge, we propose an Intelligent Evolution Controller (IEC), which is designed to evaluate whether the input instance aligns with the established central concepts or represents emerging new concepts. Furthermore, the IEC enables dynamic adjustments to the static detector by timely monitoring the uncertainty of the prediction.

Pseudo Labeling Strategy. IEC is a lightweight evidential classifier with parameters Θ_c trained with the high-confidence pseudo labels $\tilde{l}$ generated by the static detector. We categorize poorly reconstructed inputs with a label of 1, and well reconstructed inputs with a label of 0. This labeling strategy allows the IEC to discern whether incoming data matches the established central concepts in the SCD or unseen patterns.

$$\tilde{l}(\vec{x}) = \begin{cases} 1(\text{Positive}), & \text{if } L_2(\vec{x}, \vec{y}) > \mu_p \text{ and } \mathcal{U}_{ctr}(\vec{x}) \leq \mu_e \\ 0(\text{Negative}), & \text{if } L_2(\vec{x}, \vec{y}) \leq \mu_p \text{ and } \mathcal{U}_{ctr}(\vec{x}) \leq \mu_e \\ -(\text{Unknown}), & \mathcal{U}_{ctr}(\vec{x}) > \mu_e \end{cases} \quad (5)$$

where $L_2(\vec{x}, \vec{y})$ is the reconstruction error of the input instance given the static detector computed by taking the root mean squared error between $\vec{x}$ and the reconstructed output $\vec{y}$. μ_p is a predefined pseudo-labeling threshold determined by setting a proportion of the sorted reconstruction error over all training samples. $\mathcal{U}_{ctr}(\vec{x})$ is the concept uncertainty that determines the necessity of model evolution, which will be introduced in detail next. μ_e is a predefined threshold to ensure that the static detector is only trained with high-confidence samples with a low concept uncertainty $\mathcal{U}_{ctr}(\vec{x})$, i.e., $\vec{x}$ is not involved in training if the $\mathcal{U}_{ctr}(\vec{x}) > \mu_e$.

Concept Uncertainty Estimation. We introduce the predictive probability of the evidential learning model following

Eq. (2). Given the IEC parameterized by Θ_c , for a given instance $\vec{x}$, the Dirichlet-based predictive probability for class c , denoted as $\hat{p}_c = \hat{P}(y = c|\vec{x}; \Theta_c)$, is obtained by marginalizing over $\vec{p}$ as:

$$\begin{aligned} \hat{P}(y = c|\vec{x}; \Theta_c) &= \int P(y = c|\vec{p}; \Theta_c) P(\vec{p}|\vec{x}; \Theta_c) d\vec{p} \\ &= \frac{\alpha_c}{\sum_{k=1}^C \alpha_k} = \mathbb{E}_{\vec{p} \sim \text{Dir}(\vec{\alpha})} [p_c], \end{aligned} \quad (6)$$

where C denotes the number of classes, set to two in anomaly detection setting. $\vec{p}$ denotes the class probability vector modeled as a Dirichlet random variable, with p_c its component for class c , and $\vec{\alpha}$ denotes the corresponding concentration vector with component α_c produced by the evidential classifier as described in Section II-C.

IEC is trained by pseudo labels from central concepts in Eq. (5). From a Bayesian perspective, its objective can be interpreted as the negative log marginal likelihood obtained by marginalizing class probabilities under a Dirichlet prior. To address the class imbalance issue inherent in anomaly detection, we introduce a focal term that places greater emphasis on minority and misclassified examples. Concretely, our IEC objective is given by:

$$\mathcal{L}_{IEC}(\Theta_c) = (1 - \hat{p}_c)^\gamma (\log(\sum_{k=1}^C \alpha_k) - \log \alpha_c), \quad (7)$$

where $\gamma \geq 0$ is the focal exponent and $\hat{p}_c$ is the predictive probability for class c as given by Eq. (6), which is binary in anomaly detection setting. However, when the concepts of current samples differ markedly from the established central concepts, the evidence supporting these samples becomes insufficient, as the controller now lacks the information about such new concepts [21], [26]. To quantify this deviation, we propose *concept uncertainty* $\mathcal{U}_{ctr}$ defined as:

$$\begin{aligned} \mathcal{U}_{ctr}(\vec{x}; \Theta_c) &= \sum_{c=1}^C \hat{P}(y = c|\vec{x}; \Theta_c) (\Phi(\alpha_c + 1) - \\ &\Phi(\sum_{c=1}^C \alpha_c + 1)) - \sum_{c=1}^C \hat{P}(y = c|\vec{x}; \Theta_c) \log \hat{P}(y = c|\vec{x}; \Theta_c), \end{aligned} \quad (8)$$

where $\Phi(\cdot)$ is the digamma function and we utilize mutual information as a metric to evaluate the dispersion of the Dirichlet distribution across the simplex following [21].

Theoretically, the Dirichlet concentration vector $\vec{\alpha}$ quantifies the amount of evidence supporting each class prediction. When an input $\vec{x}$ conforms to the previously learned concept, the corresponding Dirichlet distribution remains concentrated, yielding low mutual information and a small $\mathcal{U}_{ctr}(\vec{x})$. In contrast, when $\vec{x}$ arises from a shifted concept, the evidence becomes diffuse and uncertain, increasing both the entropy of the predictive distribution and the mutual information term. Consequently, $\mathcal{U}_{ctr}$ rises proportionally to the model's lack of knowledge about the new concept. When $\mathcal{U}_{ctr}(\vec{x}) > \mu_e$, the static detector's prediction is deemed unreliable, prompting the dynamic shift-aware detector to reassess the input $\vec{x}$, thereby providing a theoretically grounded mechanism for identifying and responding to concept drift.

D. Dynamic Shift-aware Detector

Recall that the SCD captures central concepts from historical data but struggles to model evolving data properties accurately once concept drift occurs. Therefore, it is crucial to promptly capture emerging concepts to sustain detection accuracy over time. To this end, we introduce the Dynamic Shift-aware Detector (DSD), which enhances the SCD by enabling it to adapt to new concepts in streaming data dynamically.

Parameter Shift Measurement. We design a hypernetwork to adapt the key parameters of the static detector in DyMETER under concept drift, denoted collectively as $\Delta\Theta_d = (\Delta\Theta_d^E, \Delta\Theta_d^D)$, where $\Delta\Theta_d^E$ and $\Delta\Theta_d^D$ represent the encoder and decoder parameter shifts, respectively. Unlike the traditional hypernetwork as shown in Eq. (1) that uses a randomly initialized vector $\vec{r}^{(n)}$ to generate the model parameters, ignoring the interaction with the input instance, we replace $\vec{r}^{(n)}$ with representations derived directly from the current input to incorporate instance-specific adjustments. Concretely, the hypernetwork processes input $\vec{x}$ through a specialized subnetwork $E^{(n)}(\cdot)$ to extract relevant features $\vec{e}^{(n)}$ for adjusting parameters of each layer. Efficiency is achieved by using a shared encoder network $E_{\text{share}}(\cdot)$ across all layers, with distinct linear layers producing the necessary layer-specific vectors $\vec{e}^{(n)}$.

$$\vec{e}^{(n)} = E^{(n)}(\vec{x}) = L_{\text{layer}}^{(n)}(E_{\text{share}}(\vec{x})), \quad n = 1, \dots, N_d, \quad (9)$$

where N_d is the number of layers of the encoder and decoder, and $L_{\text{layer}}^{(n)}(\cdot)$ is the linear layer that transforms the output of $E_{\text{share}}(\cdot)$ to the features of the n -th layer. The adjustment for the n -th layer's parameters in the SCD is represented by matrix $K^{(n)} \in \mathbb{R}^{N_{in} \times N_{out}}$, correlating to the count of input and output neurons respectively. This transformation utilizes the feature vector $\vec{e}^{(n)}$, which is specific to the input, to modify the parameters of the relevant layer. In detail, this process involves the use of two MLP layers within the hypernetwork to craft these parameter changes for the n -th layer:

$$W^{(n)} = (W_1 \vec{e}^{(n)} + \vec{b}_1) W_2 + \vec{b}_2 \quad (10)$$

$$K^{(n)} = W^{(n)} + \vec{b}^{(n)} \quad (11)$$

where W_1 and W_2 are weights of the MLP layers of the hypernetwork, $\vec{b}_1$, $\vec{b}_2$ and $\vec{b}$ are the biases.

Dynamic Detector. Once the parameter shift of the static detector has been determined, we set the parameters for the dynamic detector $\Theta_d = (\Theta_d^E, \Theta_d^D)$, accordingly:

$$\Theta_d = \begin{cases} \Theta_d^E = \Theta_s^E + \Delta\Theta_d^E \\ \Theta_d^D = \Theta_s^D + \Delta\Theta_d^D \end{cases} \quad (12)$$

With the updated parameters for the dynamic detector, the reconstruction process for a given $\vec{x}$ is formulated as:

$$\vec{x} \Rightarrow \mathcal{E}_d(\vec{x}; \Theta_d^E) = \vec{z}' \Rightarrow \mathcal{D}_d(\vec{z}'; \Theta_d^D) = \vec{y}', \quad s.t. \quad \vec{x} \approx \vec{y}' \quad (13)$$

where $\vec{z}'$ is the latent representation vector and $\vec{y}'$ is the reconstructed output vector of the dynamic detector. The training process of the dynamic detector in DSD is similar to that of SCD, with the loss function $\mathcal{L}_{DSD}$ defined as follows:

$$\mathcal{L}_{DSD}(\Theta_d) = L_2(\vec{x}, \vec{y}') = \frac{\sum_{i=1}^n (\vec{x}_i - \vec{y}'_i)^2}{n} \quad (14)$$

Algorithm 1: DyMETER Training

Input : Small subset of historical stream data $\mathcal{X}_h$
Output : DyMETER parametrized by Θ_s , Θ_c and Θ_d
Initialization : Randomly initialized Θ_s , Θ_c and Θ_d

```

1 repeat
2   Randomly sample a minibatch
3   if SCD not trained in DyMETER then
4     Update  $\Theta_s$  via minimizing  $\mathcal{L}_{SCD}$  using Eq. (4)
5   else
6     Pseudo-labeling samples using Eq. (5)
7 until Convergence
8 return SCD with parameters  $\Theta_s$ 
9 repeat
10  Randomly sample a minibatch
11  Update  $\Theta_c$  via minimizing  $\mathcal{L}_{IEC}$  using Eq. (7)
12  Update parameters  $\Theta_s$ ,  $\Theta_c$ , and  $\Theta_d$  via minimizing
     $\mathcal{L}_{DSD}$  using Eq. (14)
13 until Convergence
14 return DyMETER with parameters  $\Theta_s$ ,  $\Theta_c$  and  $\Theta_d$ 

```

The training procedure of DyMETER is outlined in Algorithm 1. The SCD is first trained by minimizing $\mathcal{L}_{SCD}$ (Lines 3–4), then the IEC (Line 11) is trained using pseudo labels from the SCD. The DSD hypernetwork is subsequently optimized with gradients jointly propagated through the SCD and IEC (Line 12) to enable end-to-end adaptation.

E. Dynamic Threshold Optimization

While DyMETER adaptively models the evolving data stream, providing an evolving estimate of reconstruction errors, a fixed threshold for anomaly detection may become misaligned with the data under concept drift. To address this concern, we introduce a dynamic threshold optimization mechanism that continuously recalibrates the decision boundary for OAD based on recent observations.

Uncertainty-aware Anomaly Scoring. Let $\mathcal{R}_e$ be the reconstruction error for an incoming data sample $\vec{x}$, produced by the SCD as $L_2(\vec{x}, \vec{y})$ for static cases or DSD as $L_2(\vec{x}, \vec{y}')$ for concept drift cases. We establish the anomaly score of DyMETER considering both reconstruction error $\mathcal{R}_e$ and the concept uncertainty $\mathcal{U}_{ctr}$ as:

$$\mathcal{A}(\vec{x}) = \mathcal{R}_e \exp(\lambda \mathcal{U}_{ctr}(r_e - \mathcal{R}_e)), \quad (15)$$

where $\lambda \geq 0$ is the weight controlling how strongly uncertainty calibrates the anomaly score. r_e is a pivot value representing the referenced reconstruction error. Specifically, we initialize r_e with the maximum reconstruction error from the training set and update it via an exponential moving average (EMA) [27]. The proposed anomaly scoring mechanism affords a tunable balance, preventing the over-penalization of uncertain, large-error samples that might be normal data in unfamiliar regions and reducing the risk of overlooking uncertain, small-error instances that could be hidden anomalies.

Cost-sensitive Thresholding. Based on the anomaly scoring, we classify a sample $\vec{x}$ as anomalous if $\mathcal{A}(\vec{x}) > \mu_a$ and normal otherwise. However, maintaining the threshold μ_a as a constant proves insufficient in streaming scenarios where the ratio of anomalies may fluctuate when concept drift occurs

unpredictably. In a more comprehensive formulation, one could incorporate the distributions of anomaly score for both normal and anomalous, denoted $p_N(\mathcal{A})$ and $p_A(\mathcal{A})$, with prior probabilities π_N and π_A . By letting $\mathcal{C}_{fp}$ be the cost of a false positive and $\mathcal{C}_{fn}$ the cost of a false negative, the expected cost function takes the form as:

$$\mathcal{F}_c(\mu_a) = \pi_N \mathcal{C}_{fp} \int_{\mu_a}^{\infty} p_N(\mathcal{A}) d\mathcal{A} + \pi_A \mathcal{C}_{fn} \int_0^{\mu_a} p_A(\mathcal{A}) d\mathcal{A}, \quad (16)$$

where the first term measures the contribution from normal data misclassified as anomalies, *i.e.*, false positive rate (FPR), and the second term quantifies the contribution from the undetected anomalies, *i.e.*, false negative rate (FNR).

In principle, minimizing $\mathcal{F}_c$ yields an optimal threshold $\mu_a^* = \arg \min \mathcal{F}_c(\mu_a)$ under the specified misclassification costs and prior probabilities. However, since anomaly samples are often rare in practice, owing to the difficulty in estimating $p_A(\mathcal{A})$, we simplify threshold optimization by focusing on the distribution of reconstruction errors for data that are classified as normal. This approach is grounded in the fact that normal instances generally dominate the data stream, *i.e.*, $\pi_N \gg \pi_A$, enabling robust estimation of $p_N(\mathcal{A})$. Specifically, we maintain a sliding window $\mathcal{W}_N$ consisting of recently classified normal samples. For each arriving sample classified as normal, we include its anomaly score $\mathcal{A}$ into $\mathcal{W}_N$, and remove the oldest entry when the window reaches a predefined capacity. An empirical distribution $\hat{p}_N(\mathcal{A})$ of anomaly score is then derived from $\mathcal{W}_N$ and updated incrementally via an empirical cumulative distribution function (CDF). In particular, a histogram-based approximation of the CDF can be formulated as:

$$F_N(\mathcal{A}) = \frac{1}{|\mathcal{W}_N|} \sum_{i \in \mathcal{W}_N} \mathbb{1}_{(\mathcal{A}_i \leq \mathcal{A})} \quad (17)$$

where $\mathbb{1}_{(\cdot)}$ denotes the indicator function. By selecting a preliminary threshold μ_a^0 as the $(1 - \tau)$ -quantile of $\hat{p}_N(\mathcal{A})$, the system guarantees an upper bound on the FPR as τ . Nevertheless, controlling the FPR remains challenging when the model displays high uncertainty. To address this issue, an additional regularization term μ_a^r is introduced to fine-tune the threshold and mitigate potential misses. A second sliding window $\mathcal{W}_C$ is thus maintained, containing samples with uncertain anomaly scores close to μ_a^0 . Specifically, the anomaly score of an arriving sample is placed into $\mathcal{W}_C$ if it satisfies both $\mathcal{U}_{ctr} > \mu_t$ and $\mathcal{A} \in [\mu_a^0 - \delta, \mu_a^0 + \delta]$, where the threshold μ_t is set to the maximum concept uncertainty observed across the training set after training and δ is chosen as the median absolute deviation of $\mathcal{W}_N$. Let $\hat{\mathcal{A}}_m$ represent the median anomaly score of samples in $\mathcal{W}_C$. The regularization term μ_a^r is then computed by:

$$\mu_a^r = \kappa(\mu_a^0 - \hat{\mathcal{A}}_m) \quad (18)$$

where κ governs the strength of the regularization. Intuitively, if $\hat{\mathcal{A}}_m$ is notably less than μ_a^0 , then the overall threshold is increased to guard against potential false negatives, conversely, if $\hat{\mathcal{A}}_m$ is comparable to μ_a^0 , then μ_a^r remains small. Finally, the threshold for OAD is given by:

$$\mu_a^* = \mu_a^0 + \mu_a^r \quad (19)$$

Algorithm 2: DyMETER Inference

Input : Evolving stream data $\mathcal{X}$
Output : Anomaly scores and decisions for records in $\mathcal{X}$
Initialization : Trained SCD, IEC and DSD parametrized by Θ_s , Θ_c and Θ_d

```

1 for  $\vec{x}$  in  $\mathcal{X}$  do
2   Compute the concept uncertainty  $\mathcal{U}_{ctr}(\vec{x})$  from IEC
   using Eq. (8)
3   if  $\mathcal{U}_{ctr}(\vec{x}) > \mu_e$  then
4     Escalate to DSD using Eq. (12)
5     return Anomaly score  $\mathcal{A}(\vec{x})$  using Eq. (15)
6   else
7     Inference with SCD using Eq. (3)
8     return Anomaly score  $\mathcal{A}(\vec{x})$  using Eq. (15)
9   Calibrate threshold  $\mu_a^*$  using Eq. (19)
10  if  $\mathcal{A}(\vec{x}) > \mu_a^*$  then
11    return  $\vec{x}$  is anomaly
12  else
13    return  $\vec{x}$  is normal
14  Compute  $\mathcal{UP}$  using Eq. (20)
15  if  $\mathcal{UP} = 1$  then
16    Update DyMETER
17    break
18 return Anomaly score  $\mathcal{A}(\vec{x})$  and the decision for each  $\vec{x} \in \mathcal{X}$ 

```

When there is no significant drift, the threshold μ_a^* is updated incrementally, relying on the estimates of $\hat{p}_N(\mathcal{A})$ and exponentially decaying rank statistics $\hat{A}_m$ and δ . If the IEC signals concept drift, the DSD enhances the static detector to accommodate the new concepts in the current data stream. At the same time, the threshold optimization procedure is re-initialized to avoid using stale error statistics. Specifically, we (i) discard old entries in $\mathcal{W}_N$ and $\mathcal{W}_C$, (ii) re-estimate $\hat{p}_N(\mathcal{A})$ as new data arrives, and (iii) recompute $\mu_a^0 = \mathcal{F}_N(\mathcal{A})^{-1}(1-\tau)$ and $\mu_a^r = \kappa(\mu_a^0 - \hat{A}_m)$ to gain the final threshold μ_a^* . This ensures that both the underlying model and the decision boundary adapt in tandem, enabling the system to align well with real-world operational constraints while preserving the capacity to adapt to drifting conditions.

Offline Updating. The entire framework automatically updates its modules based on the accumulated concept uncertainty within the sliding window $\mathcal{W}_N$ over the evolving data stream. Specifically, DyMETER keeps monitoring if the concept uncertainty accumulated within this window surpasses a predefined threshold, and frequent occurrence of such events indicates that the SCD trained on the historical data streams is incapable of handling the current data stream due to increased concept drift over time. This suggests that the entire framework should be updated to align with the current data as follows:

$$\mathcal{UP} = \begin{cases} 1, & \text{if } \sum_{i=t}^{t+|\mathcal{W}_N|} (\mathbb{1}_{(\mathcal{U}_{ctr}(\vec{x}_i) > \mu_e)} \cdot \mathcal{U}_{ctr}(\vec{x}_i)) > \mu_o \text{ OR } \Delta t > T_{max} \\ 0, & \text{otherwise} \end{cases} \quad (20)$$

where $\mathcal{UP}=1$ indicates the framework should be updated, Δt is the time since the last framework update, and μ_o and T_{max} are the threshold of the framework update and maximum interval since the last update, respectively. $\sum_{i=t}^{t+|\mathcal{W}_N|} (\mathbb{1}_{(\mathcal{U}_{ctr}(\vec{x}_i) > \mu_e)} \cdot \mathcal{U}_{ctr}(\vec{x}_i))$ aggregates the concept uncertainty greater than the threshold μ_e within the current sliding window. We implement

a parallel training approach that simultaneously fine-tunes key modules offline within a sliding window and uses the latest models for OAD, enhancing efficiency and modularity without disrupting the online services. The inference procedure of DyMETER is outlined in Algorithm 2.

F. Analysis and Discussion

Effectiveness. DyMETER tackles the issue of concept drift by employing a dual-detector framework guided by the IEC. The static detector utilizes historical data to capture the central concept, which provides a stable foundation for initial anomaly detection, while the dynamic detector adjusts to new concepts by dynamically updating its parameters, ensuring responsiveness to evolving scenarios. The IEC quantifies uncertainty in the Dirichlet space, where concept drift is observed as increased distributional dispersion when the model encounters an unseen concept. Unlike entropy-based measures derived from softmax probabilities, which conflate aleatoric and epistemic uncertainty, the evidential formulation explicitly represents how confident the model is in its predictions, thereby enabling timely model adaptation. Additionally, dynamic threshold optimization within the framework allows for real-time calibration of the decision boundary, which refines the sensitivity of the anomaly detection process. Building on this, the framework also benefits from an offline updating mechanism. This further ensures ongoing detection accuracy in real-world applications.

Efficiency. Efficiency is critical in OAD for a timely response to evolving data streams. Leveraging concept uncertainty estimates from EDL, DyMETER signals drift and employs a hypernetwork to dynamically adjust model parameters, thus avoiding frequent retraining or fine-tuning of the full model, and significantly cutting computational costs while enhancing responsiveness. The time complexity of DyMETER remains lightweight and well-suited for streaming scenarios. The SCD, implemented as a symmetric autoencoder with n_S layers, incurs a cost of $\mathcal{O}(dd_h n_S)$ for an input $\vec{x}_t \in \mathbb{R}^d$ and latent dimension $d_h < d$. The IEC is implemented as an n_I -layer MLP classifier with complexity $\mathcal{O}(dn_I)$. For adaptation, the DSD employs an n_D -layer hypernetwork that generates per-instance parameter offsets, resulting in $\mathcal{O}(dn_D + |\Theta_s|)$, where $|\Theta_s|$ is the parameter size of the static detector. Ultimately, the DTO calibrates thresholds using quantile statistics over a sliding window of size $\mathcal{W}_N$, with complexity $\mathcal{O}(\mathcal{W}_N \log \mathcal{W}_N)$, while its uncertainty-based regularization introduces negligible overhead. Overall, DyMETER exhibits linear complexity in the input dimension d , with adaptation adding only the cost of generating per-instance parameter offsets of size $|\Theta_s|$, hence ensuring computational efficiency for real-time streaming data.

Interpretability. DyMETER facilitates interpretable anomaly detection by providing trustworthy uncertainty estimates essential for high-stakes scenarios, contrasting with the often unreliable softmax probabilities used by other OAD methods [8], [10], [28], [29]. Building on subjective logic (SL) theory [30], DyMETER employs EDL to explicitly model prediction reliability, which forms structured opinions under Dempster-Shafer evidence theory, mapping belief assignments to a Dirichlet distribution for nuanced anomaly detection.

Beyond reliable uncertainty quantification, DyMETER also clarifies the distinction between state changes (i.e., concept drift points) and anomalies. For example, in financial monitoring, a transition from “daily transactions” to “holiday transactions” constitutes a normal state change, whereas fraudulent transactions hidden among holiday purchases remain anomalies. DyMETER makes this separation explicit: concept drift is characterized via concept uncertainty estimation by the IEC at the input level to trigger adaptation, while anomalies are identified through reconstruction error-based anomaly scores under the current concept. This interpretability not only strengthens trust in DyMETER’s decisions but also ensures that normal state changes can be effectively tracked without being misclassified as anomalies, while genuine anomalies remain detectable under evolving distributions.

IV. EXPERIMENTS

A. Datasets

We utilize 23 benchmark datasets, comprising 19 real-world datasets from various domains and 4 synthetic datasets, designed to simulate different types of concept drift. These datasets, widely referenced in related studies, serve to evaluate the effectiveness of OAD approaches under concept drifts. We classify these datasets into two OAD settings: continuous, with temporal dependencies between observations, and discrete, with no or unknown dependencies. Data statistics for each dataset are provided in Table II.

Real-world datasets. We adopt four widely recognized anomaly detection datasets from the UCI repository and ODDS library [31], specifically the Ionosphere (Ion.), Pima, Satellite, and Mammography (Mamm.) datasets. We also incorporated the BGL dataset [32], a substantial collection of log messages from a BlueGene/L supercomputer at Lawrence Livermore National Labs, which we processed into a structured data format for analysis. Additionally, we use popular intrusion detection datasets such as KDDCUP99 [33] and NSL-KDD [34]. For time-series analysis, we draw from HexagonML’s UCR archive [35], [36], including datasets like Internal Bleeding (I.B., index 133), GaitPhase (index 129), NASA (index 158), EPG (index 145), Activity Recognition (A.R., index 162), and ECG (index 194), known for their robustness and diversity. We further include datasets like Machine Temperature and CPU utilization from the Numenta anomaly detection benchmark [37]. Lastly, we integrate the INSECTS dataset [38], which consists of optical sensor data collected during the monitoring of flying insects, to simulate concept drift with temperature level as a controlled concept. These varied datasets enabled a thorough examination of detection models under different scenarios and drift conditions.

Synthetic datasets. We adopt synthetic datasets derived from MNIST [39] and FMNIST [40], following the public benchmarks of [8]. It randomly sets categories as anomaly targets with the remaining serving as normal to simulate concepts. The occurrence locations of concept drifts are set by replacing the current concept with a new one after a random duration between one and four times the batch size. Two drift types are considered: abrupt and recurrent, where concepts switch

TABLE II: Dataset statistics.

Scenarios	Settings	Datasets	#Obj.	#Dim.	#Outliers (%)	Concept drift type
Real-world	Discrete setting	Ionosphere	351	33	126 (35.90%)	Unknown
		Pima	768	8	268 (34.90%)	Unknown
		Satellite	6435	36	2036 (31.64%)	Unknown
		Mammography	11,183	6	250 (2.32%)	Unknown
		BGL	4,713,493	9	348460 (7.39%)	Unknown
		NSL	125,973	42	58630 (46.54%)	Unknown
	Continuous setting	KDD99	494,021	41	97278 (19.69%)	Unknown
		Activity Recognition	6,675	10	59 (0.88%)	Unknown
		Internal Bleeding	7,492	10	27 (0.36%)	Unknown
		NASA	11,299	10	97 (0.86%)	Unknown
		GaitPhase	11,991	10	45 (0.38%)	Unknown
		EPG	30,000	10	50 (0.17%)	Unknown
		ECG	80,000	10	250 (0.31%)	Unknown
		Machine temperature	22,695	10	2268 (10.00%)	Unknown
		CPU utilization	18,050	10	1499 (8.30%)	Unknown
		INSECTS-Abr	44,569	33	529 (1.19%)	Abrupt
		INSECTS-Inc	48,086	33	571 (1.19%)	Incremental
		INSECTS-IncGrd	20,367	33	242 (1.19%)	Incremental/gradual
		INSECTS-IncRec	67,455	33	800 (1.19%)	Incremental/recurrent
Synthetic	Discrete setting	SynM-AbrRec	20,480	784	196 (0.96%)	Abrupt/recurrent
		SynM-GrdRec	20,480	784	192 (0.94%)	Gradual/recurrent
		SynF-AbrRec	20,480	784	193 (0.94%)	Abrupt/recurrent
		SynF-GrdRec	20,480	784	204 (1.00%)	Gradual/recurrent

sharply and seen concepts may reappear, and gradual and recurrent, where drifts occur gradually across several batches.

B. Baseline Methods

In our evaluation, we compare DyMETER with 18 baseline methods in four categories: (1) Representative anomaly detection algorithms, including Local Outlier Factor (LOF) [41], Isolation Forest (IF) [42], k-Nearest Neighbors (KNN), and STORM [43]. (2) Incremental methods, including RRCF [13], MStream [11], and MemStream [10]. (3) Ensemble methods, comprising HS-Trees [44], iForestASD [45], RS-Hash [46], LODA [47], Kitsune [9], xStream [48], PIDForest [15], and ARCUS [8]. (4) Advanced drift-adaptive architectures, namely D³R [17], SARAD [18], and our earlier work METER [4]. Overall, the baselines span diverse underlying model families, including three proximity-based (distance/density) methods, four projection-based (hashing/random projection) methods, five tree-based methods, and six neural network-based methods, as introduced below.

- **LOF** [41] is a density-based method that measures the local deviation of a point with respect to its neighbors.
- **IF** [42] is a tree-based method that isolates outliers via recursive data partitioning using decision trees.
- **KNN** is a distance-based method that identifies outliers as points with few neighbors within a specified distance.
- **STORM** [43] is a distance-based method that employs a sliding window approach to detect deviations from the window’s average behavior.
- **RRCF** [13] is a tree-based method that detects anomalies using a forest of random cut trees.
- **MStream** [11] is a hashing-based method that uses locality-sensitive hashing to identify unusual group anomalies.
- **MemStream** [10] is a neural network-based method that models streaming trends using a denoising autoencoder with a First-In-First-Out (FIFO) memory.
- **HS-Trees** [44] is a tree-based method that quickly builds its tree structure based on the dimensionality of the data space.
- **iForestASD** [45] is a tree-based method that integrates IF with sliding windows for streaming anomaly detection.
- **RS-Hash** [46] is a hashing-based method that detects anomalies using an ensemble of randomized subspace hashing and frequency-based scores.
- **LODA** [47] is a random projection-based method that employs an ensemble of one-dimensional histogram detectors for anomaly detection.

TABLE III: Performance comparison under unknown drifts in a discrete setting with higher scores indicating superior performance. We highlight the **best** and the **second best** result in each row.

Model Class	Model	Ion.		Pima		Satellite		Mamm.		BGL		NSL-KDD		KDD99	
		AUCROC	AUCPR	AUCROC	AUCPR	AUCROC	AUCPR	AUCROC	AUCPR	AUCROC	AUCPR	AUCROC	AUCPR	AUCROC	AUCPR
Traditional	LOF [41]	0.874	0.827	0.542	0.371	0.598	0.481	0.720	0.089	0.542	0.206	0.586	0.428	0.653	0.359
	IF [42]	0.860	0.817	0.677	0.502	0.676	0.375	0.867	0.211	0.823	0.295	0.530	0.577	0.784	0.406
	KNN	0.929	0.932	0.615	0.457	0.677	0.539	0.839	0.156	0.765	0.274	0.897	0.899	0.946	0.902
	STORM [43]	0.640	0.526	0.529	0.373	0.680	0.452	0.615	0.418	0.203	0.043	0.513	0.138	0.913	0.822
Incremental	RRCF [13]	0.586	0.411	0.575	0.393	0.553	0.356	0.713	0.524	0.540	0.076	0.604	0.534	0.773	0.347
	MStream [11]	0.681	0.486	0.524	0.440	0.647	0.457	0.798	0.076	0.531	0.105	0.759	0.716	0.958	0.912
	MemStream [10]	0.821	0.672	0.703	0.551	0.722	0.682	0.902	0.225	0.694	0.144	0.988	0.967	0.979	0.857
Ensemble	HS-Trees [44]	0.687	0.574	0.667	0.344	0.512	0.348	0.797	0.623	0.599	0.174	0.806	0.735	0.901	0.728
	iForestASD [45]	0.744	0.601	0.515	0.356	0.642	0.451	0.575	0.031	0.701	0.382	0.511	0.483	0.532	0.227
	RS-Hash [46]	0.743	0.502	0.518	0.372	0.640	0.586	0.776	0.622	0.436	0.245	0.684	0.524	0.783	0.707
	LODA [47]	0.514	0.373	0.501	0.347	0.500	0.316	0.500	0.023	0.523	0.074	0.504	0.535	0.507	0.197
	Kitsune [9]	0.920	0.896	0.590	0.451	0.732	0.673	0.603	0.202	0.514	0.074	0.947	0.918	0.982	0.993
	xStream [48]	0.773	0.591	0.656	0.583	0.659	0.533	0.847	0.630	0.623	0.356	0.540	0.327	0.954	0.881
	PIDForest [15]	0.821	0.718	0.669	0.474	0.718	0.543	0.847	0.202	0.791	0.300	0.503	0.561	0.864	0.772
	ARCUS [8]	0.919	0.894	0.607	0.420	0.797	0.560	0.812	0.261	0.768	0.185	0.262	0.365	0.972	0.807
Drift-adaptive	D ³ R [17]	0.758	0.605	0.546	0.363	0.731	0.577	0.748	0.112	0.872	0.436	0.910	0.817	0.947	0.758
	SARAD [18]	0.926	0.897	0.645	0.578	0.767	0.608	0.938	0.204	0.754	0.103	0.891	0.654	0.987	0.916
	METER [4]	0.950	0.956	0.733	0.654	0.796	0.777	0.913	0.491	0.895	0.369	0.982	0.963	0.973	0.853
	DyMETER	0.972	0.961	0.812	0.686	0.828	0.762	0.910	0.617	0.906	0.615	0.991	0.965	0.990	0.931

- **Kitsune** [9] is a neural network–based method that detects anomalies using an ensemble of autoencoders.
- **xStream** [48] is a random projection–based method that detects anomalies via density estimation on low-dimensional projections.
- **PIDForest** [15] is a tree-based method that trains each tree on a feature subset selected via partial identification.
- **ARCUS** [8] is a neural network–based method that employs a model pool to detect anomalies and dynamically updates its members under concept drift.
- **D³R** [17] is a neural network–based method that detects anomalies via dynamic decomposition and diffusion-based reconstruction.
- **SARAD** [18] is a neural network–based method that extracts spatial association descent via a Transformer-based autoencoder to detect anomalies.
- **METER** [4] is a neural network–based method that detects drift via evidential deep learning and adapts the detector with a static predefined decision threshold.

C. Evaluation Metrics

We utilize AUCROC and AUCPR metrics for evaluation. AUCROC calculates the area under the receiver operating characteristic (ROC) curve, plotting the false positive rate (FPR) against the true positive rate (TPR) across various thresholds. AUCPR measures the area under the precision-recall (PR) curve, displaying the relationship between precision and recall at differing thresholds. Both metrics range from 0 to 1, with higher values indicating superior performance.

D. Implementation Details

The encoder and decoder of DyMETER are implemented as symmetrically structured DNNs with 2–10 layers. The latent dimension of the autoencoder is set to the number of principal components to ensure a minimum of 70% explained variance following prior work [8], [6]. The IEC is implemented as a two-layer DNN with ReLU activations. The pseudo-labeling threshold μ_p is selected from $[0.05, 0.5]$ with a step size of 0.05, while the concept uncertainty threshold μ_e is searched within $[0.005, 0.4]$. The focal exponent γ in $\mathcal{L}_{IEC}$ is fixed to 2, and the uncertainty calibration weight λ in the anomaly score is set to 0.6. The offline update threshold μ_o is in the range

of 0.1 to 1 of the window size, and the sizes of the sliding windows $\mathcal{W}_N$ and $\mathcal{W}_C$ are each set at 64. Hyperparameters τ and κ are fixed at 0.95 and 0.8, respectively. The historical data ratio h_r is set to 0.2. DyMETER is trained using the Adam optimizer with an initial learning rate of $1e-2$, decayed exponentially by a factor of 0.96 over 2000 epochs. D³R and SARAD are implemented using the official source code released by the respective authors¹. All other baseline implementations follow the settings in our prior work [4]. We develop a data generator to simulate the generation of streaming data and report the mean performance over five independent runs. The full implementation and experimental scripts are available at <https://github.com/zjiaqi725/DyMETER>.

Experimental Environment. All the experiments are conducted in a server with Xeon(R) Silver 4114 CPU @ 2.2GHz (10 cores), 256G memory, and GeForce RTX 2080 Ti. All the models are implemented in PyTorch 1.10.0 with CUDA 10.2.

E. Effectiveness

We comprehensively evaluate DyMETER across a diverse array of datasets, comparing it against various state-of-the-art methods, with results detailed in Tables III to VI. In these experiments, DyMETER consistently outperforms competitors across different experimental conditions, data dimensions, and types of drift, highlighting its exceptional effectiveness and robustness. Notably, DyMETER exhibits significant improvements in performance over the initial version of METER in nearly all scenarios. A detailed performance analysis follows. **Comparison on real-world datasets.** We evaluate DyMETER against various baselines on 19 real-world datasets, with results detailed in Tables III, IV, and V, where DyMETER consistently excel across different drift types and settings. For instance, among the 14 results displayed in Table III, our method achieves the highest performance in 9 cases and the second highest in the remaining 4 cases among all four classes of baselines. By adopting the shingling technique for time series preprocessing as [15], [4], DyMETER effectively captures temporal dependencies, enabling DyMETER to deliver the best performance on 6 out of the 8 datasets presented in

¹D³R: <https://github.com/ForestsKing/D3R>, SARAD: <https://github.com/daidahoo/SARAD/tree/main>

TABLE IV: Performance comparison of unknown drifts in a continuous setting.

Model Class	Model	A.R.		I.B.		NASA		GaitPhase		EPG		ECG		M.T.		CPU.	
		AUCROC	AUCPR	AUCROC	AUCPR	AUCROC	AUCPR	AUCROC	AUCPR	AUCROC	AUCPR	AUCROC	AUCPR	AUCROC	AUCPR	AUCROC	AUCPR
Traditional	LOF [41]	0.615	0.170	0.782	0.180	0.653	0.477	0.821	0.328	0.934	0.679	0.670	0.016	0.501	0.141	0.560	0.112
	IF [42]	0.938	0.568	0.850	0.012	0.766	0.018	0.747	0.013	0.811	0.552	0.668	0.005	0.829	0.573	0.817	0.760
	KNN	0.508	0.132	0.540	0.165	0.475	0.478	0.532	0.200	0.083	0.001	0.247	0.002	0.759	0.255	0.724	0.452
	STORM [43]	0.597	0.313	0.534	0.072	0.593	0.041	0.602	0.137	0.578	0.436	0.662	0.523	0.604	0.127	0.667	0.605
Incremental	RRCF [13]	0.834	0.175	0.503	0.079	0.515	0.161	0.586	0.118	0.814	0.498	0.387	0.002	0.628	0.153	0.617	0.368
	MStream [11]	0.882	0.556	0.672	0.389	0.673	0.328	0.816	0.326	0.824	0.621	0.721	0.284	0.860	0.505	0.794	0.443
	MemStream [10]	0.984	0.354	0.851	0.016	0.468	0.058	0.826	0.120	0.930	0.656	0.780	0.007	0.825	0.573	0.831	0.227
Ensemble	HS-Trees [44]	0.585	0.176	0.487	0.071	0.591	0.172	0.624	0.189	0.531	0.334	0.621	0.439	0.617	0.359	0.678	0.585
	iForestASD [45]	0.517	0.202	0.641	0.137	0.511	0.086	0.689	0.208	0.782	0.470	0.733	0.006	0.738	0.231	0.755	0.153
	RS-Hash [46]	0.567	0.220	0.498	0.074	0.496	0.017	0.614	0.208	0.552	0.186	0.584	0.203	0.607	0.549	0.712	0.467
	LODA [47]	0.645	0.177	0.553	0.092	0.516	0.128	0.502	0.096	0.595	0.182	0.721	0.077	0.503	0.100	0.500	0.083
	Kitsune [9]	0.517	0.154	0.508	0.072	0.512	0.172	0.607	0.249	0.897	0.020	0.726	0.231	0.684	0.416	0.824	0.669
	xStream [48]	0.802	0.153	0.493	0.072	0.512	0.029	0.623	0.187	0.687	0.158	0.705	0.365	0.696	0.596	0.730	0.195
	PIDForest [15]	0.924	0.560	0.829	0.300	0.641	0.012	0.805	0.055	0.836	0.238	0.683	0.006	0.789	0.389	0.881	0.439
	ARCUS [8]	0.547	0.114	0.742	0.543	0.768	0.449	0.815	0.366	0.885	0.724	0.682	0.340	0.376	0.511	0.678	0.128
Drift-adaptive	D ³ R [17]	0.942	0.355	0.822	0.040	0.623	0.209	0.879	0.031	0.906	0.220	0.726	0.285	0.627	0.197	0.719	0.301
	SARAD [18]	0.926	0.584	0.889	0.392	0.712	0.354	0.821	0.285	0.934	0.521	0.686	0.135	0.871	0.665	0.847	0.692
	METER [4]	0.998	0.798	0.874	0.545	0.811	0.489	0.824	0.335	0.968	0.625	0.778	0.495	0.842	0.652	0.908	0.715
	DyMETER	0.998	0.856	0.896	0.612	0.854	0.527	0.832	0.384	0.956	0.645	0.810	0.568	0.866	0.629	0.911	0.723

TABLE V: Performance comparison of known drifts in a continuous setting in AUCROC metric.

Model Class	Model	INSECTS -Abr	INSECTS -Inc	INSECTS -IncGrd	INSECTS -IncRec	Time (s)
Traditional	LOF [41]	0.578	0.556	0.589	0.526	180
	IF [42]	0.679	0.632	0.697	0.593	67
	KNN	0.666	0.597	0.673	0.553	105
	STORM [43]	0.408	0.441	0.446	0.449	122
Incremental	RRCF [13]	0.600	0.579	0.624	0.593	121
	MStream [11]	0.703	0.698	0.788	0.672	18
	MemStream [10]	0.753	0.348	0.728	0.361	109
Ensemble	HS-Trees [44]	0.499	0.507	0.497	0.499	302
	iForestASD [45]	0.599	0.589	0.616	0.575	7985
	RS-Hash [46]	0.484	0.509	0.459	0.506	225
	LODA [47]	0.498	0.503	0.496	0.499	831
	Kitsune [9]	0.759	0.584	0.730	0.594	164
	xStream [48]	0.514	0.516	0.533	0.504	408
	PIDForest [15]	0.757	0.675	0.748	0.631	18047
	ARCUS [8]	0.601	0.597	0.576	0.632	79
Drift-adaptive	D ³ R [17]	0.776	0.786	0.713	0.743	486
	SARAD [18]	0.731	0.739	0.651	0.728	324
	METER [4]	0.816	0.795	0.712	0.794	88
	DyMETER	0.832	0.814	0.756	0.842	136

Table IV. In the continuous setting featuring various known drifts, such as abrupt and incremental, DyMETER consistently demonstrates superior performance, achieving an average improvement of 4.1% over the next suboptimal method METER. Although methods such as D³R and SARAD employ architectural designs to handle concept drift without relying on incremental updates or ensembles, and SARAD demonstrates notable performance on certain continuous datasets (e.g., M.T. and I.B.), their effectiveness is less consistent across diverse benchmarks. Furthermore, DyMETER shows remarkable computational efficiency, with inference time in Table V lower than half of the baseline methods, while providing superior performance.

Comparison on synthetic datasets. We further evaluate DyMETER on four synthetic datasets, with the outcomes detailed in Table VI. Despite the challenges posed by increased data dimensions, DyMETER consistently surpasses other baseline methods, including recent advanced drift-adaptive architectures such as D³R and SARAD, and shows marked improvements over our initial version METER. Moreover, although ARCUS occasionally reached or slightly exceeded DyMETER in the gradual drift scenarios, its performance lacks consistency. Overall, DyMETER's consistently high performance in abrupt and gradual drift scenarios, combined with reasonable computational demands, establishes it as a potent solution for OAD in dynamic environments.

Effectiveness of online and offline adaptation. To further substantiate the effectiveness of online adaptation and offline fine-tuning, we partition the INSECTS-Abr dataset into five

TABLE VI: Performance comparison of known drifts in a discrete setting in AUCROC metric.

Model Class	Model	SynM -AbrRec	SynM -GrdRec	SynF -AbrRec	SynF -GrdRec	Time (s)
Traditional	LOF [41]	0.554	0.539	0.473	0.503	620
	IF [42]	0.565	0.544	0.497	0.478	328
	KNN	0.550	0.584	0.515	0.513	55
	STORM [43]	0.513	0.516	0.521	0.530	323
Incremental	RRCF [13]	0.695	0.666	0.681	0.715	142
	MStream [11]	0.473	0.608	0.623	0.507	31
	MemStream [10]	0.517	0.496	0.523	0.472	549
Ensemble	HS-Trees [44]	0.497	0.503	0.500	0.511	139
	iForestASD [45]	0.523	0.514	0.533	0.506	2580
	RS-Hash [46]	0.482	0.506	0.470	0.506	102
	LODA [47]	0.504	0.500	0.502	0.500	328
	Kitsune [9]	0.544	0.502	0.570	0.523	323
	xStream [48]	0.641	0.658	0.625	0.605	997
	PIDForest [15]	0.514	0.546	0.513	0.503	8176
	ARCUS [8]	0.901	0.894	0.759	0.787	120
Drift-adaptive	D ³ R [17]	0.761	0.699	0.672	0.619	508
	SARAD [18]	0.693	0.633	0.757	0.670	368
	METER [4]	0.879	0.825	0.806	0.632	72
	DyMETER	0.906	0.886	0.858	0.728	142

sequential and equal subsets, denoted as p_1 to p_5 . Then, we further split each subset into one training set and one test set, and derive $p_{1-train}/p_{1-test}$ to $p_{5-train}/p_{5-test}$ correspondingly. Using these subsets, we conduct two groups of experiments. In Group I, we train DyMETER on $p_{1-train}$ and then fine-tune DyMETER on $p_{2-train}$ to $p_{5-train}$ respectively, and report the detection performance of DyMETER. As for the experiments of Group II, we only train DyMETER on $p_{1-train}$ once, and then report the trained DyMETER on all test sets, namely p_{1-test} to p_{5-test} . Results summarized in Table VII show that (i) offline adaptation (Group I) can even degrade performance compared to the model trained once with only online adaptation, as observed on p_{4-test} and p_{5-test} , and (ii) the model with online adaptation (Group II) achieves an average AUCROC of 0.795, which is comparable to the average performance of Group I with costly offline fine-tuning and only slightly lower than 0.811 obtained on p_{1-test} . These findings suggest that offline adaptation is not always effective for OAD tasks, which may be attributed to the information bottleneck inherent in reconstruction-based models: if it is too restrictive, the model *underfits* and fails to reconstruct even normal data, whereas if it is too loose, the model *overfits* and reconstructs anomalies successfully.

F. Ablation Study

In this section, we conduct ablation studies to evaluate both the effectiveness of individual components and the contributions of their dedicated mechanism designs under diverse and realistic drift scenarios, as detailed below.

TABLE VII: Performance comparison of DyMETER with offline and online adaptation measured in AUCROC.

Variant	$p1\text{-test}$	$p2\text{-test}$	$p3\text{-test}$	$p4\text{-test}$	$p5\text{-test}$	Average
Group I	0.811	0.802	0.780	0.792	0.795	0.796 ± 0.010
Group II	0.811	0.783	0.766	0.812	0.802	0.795 ± 0.018

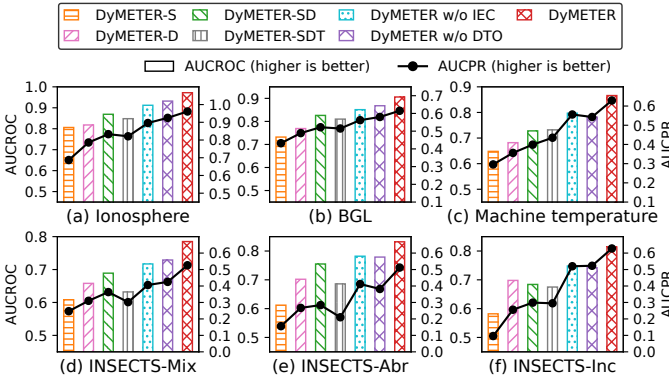


Fig. 3: Ablation analysis on module effectiveness via AUCROC and AUCPR for DyMETER and its six variants.

Module effectiveness. We evaluate the contribution of each DyMETER module on six real-world datasets subject to diverse and often unknown concept drifts. We define six DyMETER variants: DyMETER-S and DyMETER-D, which include only the SCD or DSD module, respectively; DyMETER-SD and DyMETER-SDT retain SCD with DSD or DTO, respectively, while excluding all other modules; and DyMETER w/o IEC and DyMETER w/o DTO, which respectively omit the IEC or DTO module from DyMETER. The results in Figure 3 consistently confirm that each module contributes considerably to enhancing detection performance under concept drift. Notably, the results indicate that DSD plays a critical role, with its removal leading to up to an 18.1% drop in AUCROC across the evaluated datasets, and its integration with SCD, which provides a stable basis for adaptation, further improves overall effectiveness. In addition, both IEC and DTO are important for effective online anomaly detection. IEC identifies drift-affected instances to trigger adaptation only when necessary, while DTO continuously recalibrates the decision boundary as concepts evolve and yields a 7.1% AUCROC gain across the evaluated datasets.

Backbone Flexibility. We conduct extensive evaluation of various SCD backbone designs to examine the flexibility of DyMETER, including LSTM, 1D convolution, and 1D convolution with residual connections, denoted as DyMETER-lstm, DyMETER-conv, and DyMETER-r-conv, respectively. For a fair comparison, all variants share the same base structure as DyMETER’s DNNs implementation. Results in Table VIII show that all three enhanced SCD backbones achieve consistently better detection performance, yielding AUCROC improvements of 4.4%, 3.1%, and 3.8%, respectively, compared to the original SCD based on canonical DNNs. These results confirm that DyMETER’s effectiveness is not tied to a specific backbone architecture, demonstrating its architecture-agnostic design and flexibility for further performance improvements.

Focal objective function. To assess the effectiveness of the focal objective function $\mathcal{L}_{IEC}$, we introduce DyMETER-

TABLE VIII: Performance on dedicated ablation studies measured by AUCROC.

Variant	Ion.	BGL	M.T.	INSECTS -Mix	INSECTS -Abr	INSECTS -Inc
DyMETER-lstm	0.976	0.925	0.912	0.815	0.918	0.859
DyMETER-conv	0.982	0.924	0.893	0.811	0.889	0.838
DyMETER-r-conv	0.986	0.932	0.892	0.807	0.916	0.841
DyMETER-ce	0.961	0.898	0.852	0.739	0.802	0.787
DyMETER-pl	0.982	0.932	0.872	0.802	0.842	0.831
DyMETER-re	0.931	0.816	0.725	0.628	0.682	0.536
DyMETER-R	0.948	0.860	0.842	0.755	0.817	0.798
DyMETER-Q	0.962	0.864	0.846	0.761	0.829	0.806
DyMETER-A	0.967	0.877	0.871	0.769	0.818	0.810
DyMETER	0.972	0.906	0.866	0.785	0.832	0.814

TABLE IX: Training and inference efficiency of DyMETER.

Dataset	Throughput Training	Throughput Inference	Training Time (s)	Memory (MiB)
Ion.	8,962	165,120	0.009	6.20
NSL	140,662	77,893,957	0.022	16.28
M.T.	300,918	2,068,214	0.018	9.71
CPU	205,884	4,300,618	0.016	9.22
INSECTS-Abr	520,218	21,576,902	0.017	38.08

ce, a variant trained with cross-entropy loss for the IEC module. Comparing DyMETER-ce with DyMETER reveals that incorporating the focal term in the loss function yields an average 2.7% performance improvement across six datasets in Table VIII. Given the inherent class imbalance in anomaly detection tasks and the imbalance in our pseudo labels, the focal term enhances IEC training, leading to superior performance. **Prior knowledge.** We introduce a variant DyMETER-pl, which enhances the pseudo-labeling strategy by incorporating 1% of labeled anomalies into the training set to evaluate DyMETER’s adaptability and generalization in situations with limited labeled samples. The results in Table VIII reveal that the evidential IEC successfully leverages prior knowledge and considerably enhances the learning capacity of DyMETER by incorporating only a small number of labeled samples.

Instance-specific information. A dedicated ablation study is conducted to evaluate the superiority of the instance-aware hypernetwork in DSD, where parameter generation is explicitly conditioned on the current input instance, as opposed to using conventional random embeddings (denoted as DyMETER-re). As shown in Table VIII, incorporating instance-specific information significantly improves DyMETER’s detection performance. In contrast to the original hypernetwork design, which relies on random embeddings and thus exhibits a weak correlation between parameter generation and the current input, DSD takes into account instance-specific information, enabling more precise and adaptive modeling for online anomaly detection under concept drift.

Dynamic thresholding. To analyze the mechanism design underlying the proposed dynamic thresholding in DTO, we introduce three refined variants with the results summarized in Table VIII. DyMETER-R relies solely on reconstruction error for anomaly scoring, resulting in notable performance deterioration. This highlights the crucial role of concept uncertainty in refining anomaly assessment by calibrating the anomaly degree based on reconstruction error. DyMETER-Q adopts quantile-based dynamic thresholding μ_a^0 , guaranteeing an upper bound on the false positive rate. However, it be-

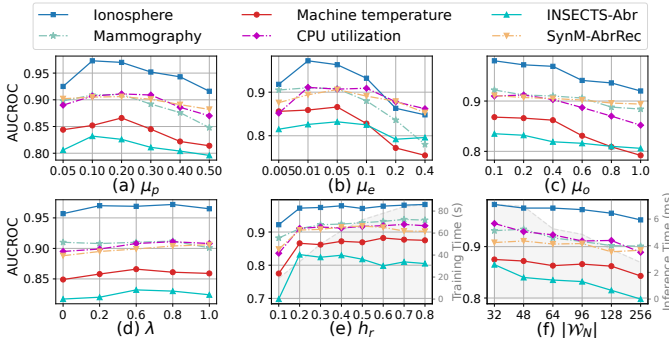


Fig. 4: Sensitivity test of key hyperparameters.

comes overly conservative under high uncertainty, resulting in degraded overall performance due to increased false negatives, which underscores the necessity of the uncertainty-aware regularization term in DTO. DyMETER- $\mathcal{A}$ replaces the median anomaly score of the sliding window $\mathcal{W}_C$ with the mean to calculate the regularization term μ_a^r . Although it achieves average performance comparable to DyMETER, it degrades on five out of six datasets, indicating that the median-based design is more robust for threshold regularization under concept drift.

G. Efficiency

We evaluate DyMETER's training and inference efficiency by quantifying its throughput, defined as the number of samples processed per second, across benchmark datasets. As shown in Table IX, DyMETER maintains high efficiency in both training and inference phases across datasets of varying sizes and dimensions, demonstrating its computational effectiveness and supporting rapid response for OAD applications.

Another important consideration is the training time and the peak memory usage. Table IX reports the average training time for each epoch and the peak memory. The results demonstrate that DyMETER requires negligible training time and takes low memory usage across these datasets. This is mainly due to the lightweight design of the key modules of DyMETER, as discussed in detail in Section III-F. To provide further insights into this matter, we conduct tests to assess the efficiency impact of the IEC and DSD modules. Specifically, we compare peak memory usage and average training time per epoch on CPU of DyMETER with and without IEC and DSD. The results indicate that the introduction of the IEC and DSD modules leads to a negligible increase in training time by 0.014s and 0.007s, respectively, and peak memory usage by 5.17MB and 2.61MB, respectively.

H. Sensitivity Study

We first conduct a sensitivity study on three threshold hyperparameters μ_p , μ_e , and μ_o . As shown in Figure 4, the performance curves of the pseudo-labeling threshold μ_p and uncertainty threshold μ_e across six datasets exhibit an initial increase followed by a gradual decline. This pattern arises because setting these thresholds too low assigns an excessive number of samples to the DSD module even under stable concepts, whereas overly high thresholds delay or suppress

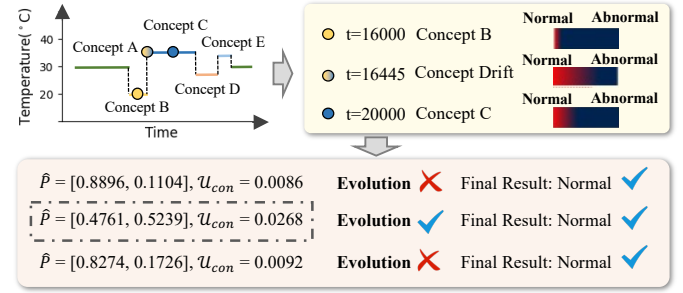


Fig. 5: Interpretation of model behavior across evolving concepts.

timely adaptation to concept drift. The parameter μ_o , representing the ratio of samples exceeding the update threshold within the sliding window $\mathcal{W}_N$, shows mild fluctuations but remains generally stable. Smaller μ_o values improve detection accuracy but may trigger frequent offline updates, thereby reducing efficiency. In practice, when prior data characteristics are unknown, we recommend setting μ_p between 0.1 and 0.2, μ_e within 0.01–0.05, and μ_o between 0.2 and 0.4 to balance detection performance and computational cost.

Additionally, we analyze the sensitivity of λ , which controls the strength of uncertainty calibration in the anomaly score. The results show mild sensitivity across all datasets, with performance rising initially and slightly declining afterward, and λ values between 0.6 and 0.8 yielding the most consistent results. We further evaluate the historical data ratio h_r within the range of 0.1 to 0.8 and observe that low ratios lead to notable performance degradation due to insufficient information for learning central concepts. Performance improves as h_r increases, whereas overly large ratios bias the model toward outdated concepts, reducing adaptability to new concepts and increasing training cost. To balance effectiveness and efficiency, we set $h_r = 0.2$ in this work. Similarly, the window size $|\mathcal{W}_N|$ critically balances accuracy and efficiency in DTO. As shown in Figure 4, smaller windows ensure timely updates but increase inference time, while larger ones reduce cost at the expense of slower adaptation. Overall, DyMETER maintains stable performance across a broad range of $|\mathcal{W}_N|$, enabling flexible adjustment for different practical needs.

I. Interpretability

To provide a semantic interpretation, Figure 5 illustrates DyMETER's behavior across evolving concepts, showing how probability shifts and uncertainty dynamics drive adaptation. At $t = 16000$, DyMETER exhibits low entropy and uncertainty, correctly recognizing the sample as normal. At $t = 16445$, high concept uncertainty indicates a distributional shift, activating the IEC's dynamic mode. At $t = 16445$, DyMETER yields a probability distribution marked by significant concept uncertainty, suggesting a potential for concept drift that warrants attention. In response, DyMETER transitions to the dynamic mode and adjusts base parameters in real time, accurately classifying the sample as normal. By $t = 20000$, we can observe that the model has effectively adapted to the new concept, as evidenced by low uncertainty in its predictions. These results show that DyMETER achieves interpretable decisions while

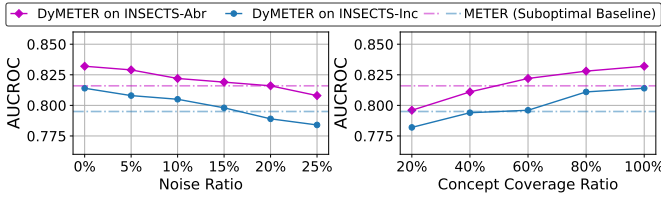


Fig. 6: Robustness under noisy and insufficient concept coverage.

decoupling drift detection from anomaly identification and tracking state changes via uncertainty estimation.

J. Robustness

To further examine the practical robustness of DyMETER, we simulate two challenging yet realistic scenarios: (i) Noisy initialization, where we inject Gaussian noise into training while keeping the training set size unchanged, mimicking environments with unstable or mislabeled records, and (ii) Insufficient concept coverage, where only a subset of the original training set is used, limiting the representation of the nominal concept. The results summarized in Figure 6 span representative drift settings, abrupt and incremental drift, evaluated on the INSECTS-Abr and INSECTS-Inc, respectively. Although performance slightly declines, the degradation remains modest and smooth, and DyMETER still consistently outperforms the suboptimal baseline (METER) over a wide range of noise and coverage levels, confirming robustness even under imperfect initial concept learning. The robustness of DyMETER stems from its hierarchical and uncertainty-aware training design, where IEC is trained only on low-uncertainty samples to prevent noise propagation as Eq. (5), while DTO continuously recalibrates decision boundaries to absorb score shifts and sustain stability, with offline updates providing a contingency mechanism under extreme conditions.

K. In-depth Analysis on Concept Drift Adaptation

To gain deeper insight into DyMETER's behavior in evolving environments, we perform an in-depth analysis of its dynamic performance across different drift scenarios. First, we visualize the temporal evolution of AUCROC and concept uncertainty along the data stream under abrupt and incremental drift settings, where changes in temperature serve as drift indicators following [38]. As illustrated in Figure 7, DyMETER consistently exhibits high AUCROC without notable degradation throughout the data stream. Although the uncertainty curve exhibits mild fluctuations, it rises sharply at drift onset with virtually no delay, validating the model's responsiveness to evolving concepts. To further assess adaptability under more complex conditions, we construct a large-scale dataset with mixed drift types by sequentially concatenating four INSECTS variants. This setup produces a unified data stream where multiple drift types (i.e., abrupt, incremental, gradual, and reoccurring) co-exist. To ensure label-space consistency and smooth transitions between adjacent data streams, mixed sampling is applied at their junctions. As shown in Figure 7(c), although the AUCROC exhibits slight declines over time (e.g., around $t=40000$ and $t=100000$), the sharp rise in uncertainty at

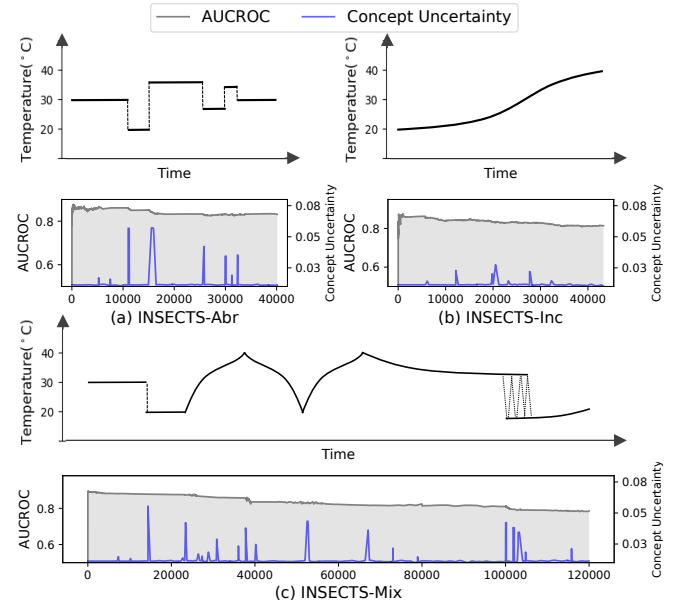


Fig. 7: In-depth analysis on concept drift adaptation.

drift onsets promptly activates DSD for online adaptation, enabling DyMETER to sustain competitive overall performance (AUCROC above 0.785) across complex drift dynamics.

L. DyMETER in Action

To demonstrate the practical application of DyMETER, we integrated it into Apache Flink, a sophisticated framework for stateful computations across both unbounded and bounded data streams [49]. DyMETER operates within a dedicated Flink operator that not only manages the necessary computational environment and resources but also facilitates data interactions through versatile connectors and ensures robust failure management. This integration enables DyMETER to support real-time data processing pipelines, perform large-scale exploratory data analysis, and execute efficient ETL processes. Our experiments on the INSECT-Abr dataset showcase DyMETER's seamless integration within Flink in a live setting, as shown in Figure 8. Notably, DyMETER consistently achieves a high AUCROC level, showcasing its reliability for ongoing and effective anomaly detection in real-time operational environments.

V. RELATED WORK

Anomaly Detection. Anomaly detection (AD) has been extensively studied in various fields such as intrusion detection [50], [51], healthcare [52], [53], [54], and finance [55], [56]. With the rise of deep learning, reconstruction-based approaches leveraging autoencoders [57], [51], variational autoencoders (VAEs) [58], [54], and generative adversarial networks (GANs) [59] have become dominant, detecting anomalies as instances with large reconstruction errors. More recently, diffusion-based models [60], [61], [62] have emerged, further advancing AD through superior reconstruction quality. However, their reliance on extensive training data and inability to adapt to evolving data streams make them unsuitable for online anomaly detection.

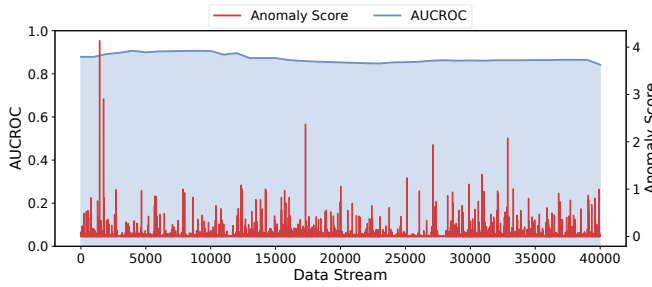


Fig. 8: Timely performance of DyMETER on Flink.

Online Anomaly Detection. Online anomaly detection (OAD) aims to identify abnormal behavior in evolving data streams. Unlike traditional AD, OAD faces the challenge of concept drift. Early studies [63], [64], [65] primarily rely on sliding windows to monitor statistical shifts (e.g., mean or variance) in streaming features. However, such methods are insufficient for addressing the more intricate distributional shifts, correlations, and pattern transformations that characterize real-world drifts. To handle evolving streams, various AD paradigms have been extended for online use, giving rise to two dominant OAD categories: incremental and ensemble methods.

Incremental methods [13], [66], [11], [10], [14] construct an initial detector and progressively update it as new data arrives. Several algorithms rely on similarity measures to detect anomalies in evolving sequences [13], [14], while deep learning models extend this paradigm by capturing non-linear feature dependencies for richer representations. For instance, MStream [11] and its deep variant MStream-AE employ autoencoder-based representations to improve detection accuracy, while MemStream [10] integrates a denoising autoencoder with a memory module to model evolving data distributions. However, incremental methods remain computationally inefficient and prone to adaptation delays, as they require frequent model updates through retraining or fine-tuning to accommodate new concepts.

Ensemble methods [44], [45], [46], [47], [48], [8], [9], [15] maintain multiple models specialized for different concepts and aggregate their outputs for robust detection under evolving distributions. iForestASD [45] combines sliding windows with isolation forests to detect anomalies in streaming data, while xStream [48] employs an ensemble of randomized projections that are continuously updated to accommodate temporal shifts. Beyond these methods, deep learning-driven ensembles have further advanced this paradigm. For instance, Kitsune [9] employs multiple autoencoders to collaboratively detect anomalies in network streams, and ARCUS [8] introduces adaptive model pooling to manage multiple deep models for time-varying concept drifts. However, the effectiveness of ensemble methods is constrained by the number and diversity of constituent models, whose maintenance incurs considerable computational and memory overhead.

Beyond these paradigms, recent studies have proposed drift-adaptive architectures that embed adaptation mechanisms directly into model design [17], [18], [19], [20], [67]. For instance, D³R [17] addresses drift through dynamic decomposition and diffusion-based reconstruction, while SARAD [18]

employs Transformer-based spatial-temporal association modeling to capture evolving inter-feature relationships. While these approaches alleviate the reliance on repeated tuning and model ensembles, they still lack online fine-grained adaptability and robust threshold calibration under evolving concepts. In contrast, DyMETER advances this line of research by integrating instance-aware inference-time adaptation with dynamic threshold optimization, enabling effective, efficient, and interpretable anomaly detection across diverse drift scenarios.

VI. LIMITATIONS AND FUTURE DIRECTIONS

While DyMETER demonstrates strong adaptability and robustness across diverse drift scenarios, several limitations remain that open avenues for future research. The current framework assumes that historical data provide a reasonably representative basis for the underlying nominal concept. In highly noisy or data-scarce environments, this assumption may not always hold, potentially leading to an imperfect initialization. Nevertheless, the robustness experiments in Section IV-H show that DyMETER remains resilient under such adverse conditions, consistently achieving competitive performance against the suboptimal baseline. Future extensions could incorporate self-supervised or regularization strategies to further enhance initialization robustness and reduce reliance on clean historical data. Another promising direction is to advance DyMETER toward proactive adaptation, where predictive uncertainty dynamics are leveraged to anticipate concept drift events before they occur, ultimately enabling lifelong anomaly detection in evolving real-world environments.

VII. CONCLUSIONS

In this paper, we propose a novel online anomaly detection (OAD) framework DyMETER that effectively, efficiently, and interpretably addresses the concept drift challenge in evolving data streams. DyMETER integrates a static detector, trained on historical data, to capture recurring central concepts, while dynamically adapting to emerging ones through a hypernetwork-based dynamic detector. To achieve efficient and interpretable model evolution, an intelligent evolution controller is introduced to estimate concept uncertainty and guide adaptive updates. Furthermore, a dynamic threshold optimization strategy enables adaptive decision boundaries that continuously adjust to emerging concepts in real time. Extensive experimental evaluations demonstrate that DyMETER consistently outperforms state-of-the-art OAD methods across diverse scenarios while facilitating valuable interpretability.

REFERENCES

- [1] C. Qiu, M. Kloft, S. Mandt, and M. Rudolph, "Self-supervised anomaly detection with neural transformations," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.
- [2] G. Lu, F. Zhou, M. Pavlovski, C. Zhou, and C. Jin, "A robust prioritized anomaly detection when not all anomalies are of primary interest," in *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 2024, pp. 775–788.
- [3] W. Guan, J. Cao, H. Zhao, Y. Gu, and S. Qian, "Wake: A weakly supervised business process anomaly detection framework via a pre-trained autoencoder," *IEEE Transactions on Knowledge and Data Engineering*, 2023.

- [4] J. Zhu, S. Cai, F. Deng, B. C. Ooi, and W. Zhang, "Meter: A dynamic concept adaptation framework for online anomaly detection," *Proceedings of the VLDB Endowment*, vol. 17, no. 4, pp. 794–807, 2023.
- [5] T. Xiang, Y. Zhang, Y. Lu, A. Yuille, C. Zhang, W. Cai, and Z. Zhou, "Exploiting structural consistency of chest anatomy for unsupervised anomaly detection in radiography images," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.
- [6] K. H. Kim, S. Shim, Y. Lim, J. Jeon, J. Choi, B. Kim, and A. S. Yoon, "Rapp: Novelty detection with reconstruction along projection pathway," in *International Conference on Learning Representations*, 2020.
- [7] C.-H. Lai, D. Zou, and G. Lerman, "Robust subspace recovery layer for unsupervised anomaly detection," *arXiv preprint arXiv:1904.00152*, 2019.
- [8] S. Yoon, Y. Lee, J.-G. Lee, and B. S. Lee, "Adaptive model pooling for online deep anomaly detection from a complex evolving data stream," in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022, pp. 2347–2357.
- [9] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *25th Annual Network and Distributed System Security Symposium, NDSS 2018*. The Internet Society, 2018.
- [10] S. Bhatia, A. Jain, S. Srivastava, K. Kawaguchi, and B. Hooi, "Memstream: Memory-based streaming anomaly detection," in *Proceedings of the ACM Web Conference 2022*, 2022, pp. 610–621.
- [11] S. Bhatia, A. Jain, P. Li, R. Kumar, and B. Hooi, "Mstream: Fast anomaly detection in multi-aspect streams," in *Proceedings of the Web Conference 2021*, 2021, pp. 3371–3382.
- [12] S. Yoon, Y. Shin, J.-G. Lee, and B. S. Lee, "Multiple dynamic outlier-detection from a data stream by exploiting duality of data and queries," in *Proceedings of the 2021 International Conference on Management of Data*, 2021, pp. 2063–2075.
- [13] S. Guha, N. Mishra, G. Roy, and O. Schrijvers, "Robust random cut forest based anomaly detection on streams," in *International conference on machine learning*. PMLR, 2016, pp. 2712–2721.
- [14] P. Boniol, J. Paparrizos, T. Palpanas, and M. J. Franklin, "Sand: streaming subsequence anomaly detection," *Proceedings of the VLDB Endowment*, vol. 14, no. 10, pp. 1717–1729, 2021.
- [15] P. Gopalan, V. Sharan, and U. Wieder, "Pidforest: anomaly detection via partial identification," *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [16] T. Kieu, B. Yang, C. Guo, and C. S. Jensen, "Outlier detection for time series with recurrent autoencoder ensembles," in *IJCAI*, 2019, pp. 2725–2732.
- [17] C. Wang, Z. Zhuang, Q. Qi, J. Wang, X. Wang, H. Sun, and J. Liao, "Drift doesn't matter: Dynamic decomposition with diffusion reconstruction for unstable multivariate time series anomaly detection," *Advances in neural information processing systems*, vol. 36, pp. 10758–10774, 2023.
- [18] Z. Dai, L. He, S. Yang, and M. Leeke, "Sarad: Spatial association-aware anomaly detection and diagnosis for multivariate time series," *Advances in Neural Information Processing Systems*, vol. 37, pp. 48371–48410, 2024.
- [19] B. Li, S. Gupta, and E. Müller, "State-transition-aware anomaly detection under concept drifts," *Data & Knowledge Engineering*, vol. 154, p. 102365, 2024.
- [20] D. Kim, S. Park, and J. Choo, "When model meets new normals: Test-time adaptation for unsupervised time-series anomaly detection," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 38, no. 12, 2024, pp. 13 113–13 121.
- [21] M. Sensoy, L. Kaplan, and M. Kandemir, "Evidential deep learning to quantify classification uncertainty," *Advances in neural information processing systems*, vol. 31, 2018.
- [22] X. Ma, F. Liu, J. Wu, J. Yang, S. Xue, and Q. Z. Sheng, "Rethinking unsupervised graph anomaly detection with deep learning: Residuals and objectives," *IEEE Transactions on Knowledge and Data Engineering*, 2024.
- [23] A. Abdulaal, Z. Liu, and T. Lancewicki, "Practical approach to asynchronous multivariate time series anomaly detection and localization," in *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*, 2021, pp. 2485–2494.
- [24] M. S. Graham, W. H. Pinaya, P.-D. Tudosiu, P. Nachev, S. Ourselin, and J. Cardoso, "Denoising diffusion models for out-of-distribution detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 2948–2957.
- [25] M. Haroush, T. Frostig, R. Heller, and D. Soudry, "Statistical testing for efficient out of distribution detection in deep neural networks," *arXiv preprint arXiv:2102.12967*, 2021.
- [26] K. W. Ng, G.-L. Tian, and M.-L. Tang, "Dirichlet and related distributions: Theory, methods and applications," 2011.
- [27] F. Klinker, "Exponential moving average versus moving exponential average," *Mathematische Semesterberichte*, vol. 58, pp. 97–107, 2011.
- [28] A. Ntroumpogiannis, M. Giannoulis, N. Myrtakis, V. Christophides, E. Simon, and I. Tsamardinos, "A meta-level analysis of online anomaly detectors," *The VLDB Journal*, vol. 32, no. 4, pp. 845–886, 2023.
- [29] K. Bountrogiannis, G. Tzagkarakis, and P. Tsakalides, "Distribution agnostic symbolic representations for time series dimensionality reduction and online anomaly detection," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 6, pp. 5752–5766, 2022.
- [30] A. Jsang, *Subjective Logic: A formalism for reasoning under uncertainty*. Springer Publishing Company, Incorporated, 2018.
- [31] S. Rayana, "Odds library," 2016, <https://odds.cs.stonybrook.edu>. Accessed:2023-07.
- [32] A. Oliner and J. Stearley, "What supercomputers say: A study of five system logs," in *37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN'07)*, 2007, pp. 575–584.
- [33] "Kdd cup dataset," <http://kdd.ics.uci.edu/databases/kddcup99/kddcup99.html>. Accessed:2023-07, 1999.
- [34] M. Tavallae, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the kdd cup 99 data set," in *2009 IEEE symposium on computational intelligence for security and defense applications*. Ieee, 2009, pp. 1–6.
- [35] H. A. Dau, E. Keogh, K. Kamgar, C.-C. M. Yeh, Y. Zhu, S. Gharghabi, C. A. Ratanamahatana, Yanping, B. Hu, N. Begum, A. Bagnall, A. Mueen, and G. Batista, "The ucr time series classification archive," October 2021, https://www.cs.ucr.edu/~eamonn/time_series_data_2018/UCR_TimeSeriesAnomalyDatasets2021.zip. Accessed:2023-07.
- [36] R. Wu and E. J. Keogh, "Current time series anomaly detection benchmarks are flawed and are creating the illusion of progress," *IEEE transactions on knowledge and data engineering*, vol. 35, no. 3, pp. 2421–2429, 2021.
- [37] S. Ahmad, A. Lavin, S. Purdy, and Z. Agha, "Unsupervised real-time anomaly detection for streaming data," *Neurocomputing*, vol. 262, pp. 134–147, 2017.
- [38] V. M. Souza, D. M. dos Reis, A. G. Maletzke, and G. E. Batista, "Challenges in benchmarking stream learning algorithms with real-world data," *Data Mining and Knowledge Discovery*, vol. 34, pp. 1805–1858, 2020.
- [39] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 2002.
- [40] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms," *arXiv preprint arXiv:1708.07747*, 2017.
- [41] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "Lof: identifying density-based local outliers," in *Proceedings of the 2000 ACM SIGMOD international conference on Management of data*, 2000, pp. 93–104.
- [42] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *2008 eighth IEEE international conference on data mining*. IEEE, 2008, pp. 413–422.
- [43] F. Angiulli and F. Fassetto, "Detecting distance-based outliers in streams of data," in *Proceedings of the sixteenth ACM conference on Conference on information and knowledge management*, 2007, pp. 811–820.
- [44] S. C. Tan, K. M. Ting, and T. F. Liu, "Fast anomaly detection for streaming data," in *Twenty-second international joint conference on artificial intelligence*. Citeseer, 2011.
- [45] Z. Ding and M. Fei, "An anomaly detection approach based on isolation forest algorithm for streaming data using sliding window," *IFAC Proceedings Volumes*, vol. 46, no. 20, pp. 12–17, 2013.
- [46] S. Sathé and C. C. Aggarwal, "Subspace outlier detection in linear time with randomized hashing," in *2016 IEEE 16th International Conference on Data Mining (ICDM)*. IEEE, 2016, pp. 459–468.
- [47] T. Pevný, "Loda: Lightweight on-line detector of anomalies," *Machine Learning*, vol. 102, pp. 275–304, 2016.
- [48] E. Manzoor, H. Lamba, and L. Akoglu, "xstream: Outlier detection in feature-evolving data streams," in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2018, pp. 1963–1972.
- [49] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi, and K. Tzoumas, "Apache flink: Stream and batch processing in a single engine," *The Bulletin of the Technical Committee on Data Engineering*, vol. 38, no. 4, 2015.
- [50] K. Sood, M. R. Nosouhi, D. D. N. Nguyen, F. Jiang, M. Chowdhury, and R. Doss, "Intrusion detection scheme with dimensionality reduction in next generation networks," *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 965–979, 2023.

- [51] J. Zhu, F. Deng, J. Zhao, and J. Chen, "Adaptive aggregation-distillation autoencoder for unsupervised anomaly detection," *Pattern Recognition*, vol. 131, p. 108897, 2022.
- [52] J. Su, H. Shen, L. Peng, and D. Hu, "Few-shot domain-adaptive anomaly detection for cross-site brain images," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 3, pp. 1819–1835, 2021.
- [53] E. Šabić, D. Keeley, B. Henderson, and S. Nannemann, "Healthcare and anomaly detection: using machine learning to predict anomalies in heart rate data," *AI & SOCIETY*, vol. 36, no. 1, pp. 149–158, 2021.
- [54] J. Zhu, F. Deng, J. Zhao, D. Liu, and J. Chen, "Uaed: Unsupervised abnormal emotion detection network based on wearable mobile device," *IEEE Transactions on Network Science and Engineering*, vol. 10, no. 6, pp. 3682–3696, 2023.
- [55] F. Xiao, S. Cai, G. Chen, H. Jagadish, B. C. Ooi, and M. Zhang, "Vecaug: Unveiling camouflaged frauds with cohort augmentation for enhanced detection," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024, pp. 6025–6036.
- [56] W. Hilal, S. A. Gadsden, and J. Yawney, "Financial fraud: A review of anomaly detection techniques and recent advances," 2022.
- [57] H. S. Vu, D. Ueta, K. Hashimoto, K. Maeno, S. Pranata, and S. M. Shen, "Anomaly detection with adversarial dual autoencoders," *arXiv preprint arXiv:1902.06924*, 2019.
- [58] L. Li, J. Yan, H. Wang, and Y. Jin, "Anomaly detection of time series with smoothness-inducing sequential variational auto-encoder," *IEEE transactions on neural networks and learning systems*, vol. 32, no. 3, pp. 1177–1191, 2020.
- [59] Z. Zhang, W. Li, W. Ding, L. Zhang, Q. Lu, P. Hu, T. Gui, and S. Lu, "Stad-gan: unsupervised anomaly detection on multivariate time series with self-training generative adversarial networks," *ACM Transactions on Knowledge Discovery from Data*, vol. 17, no. 5, pp. 1–18, 2023.
- [60] Y. Chen, C. Zhang, M. Ma, Y. Liu, R. Ding, B. Li, S. He, S. Rajmohan, Q. Lin, and D. Zhang, "Imdiffusion: Imputed diffusion models for multivariate time series anomaly detection," *Proceedings of the VLDB Endowment*, vol. 17, no. 3, pp. 359–372, 2023.
- [61] H. He, J. Zhang, H. Chen, X. Chen, Z. Li, X. Chen, Y. Wang, C. Wang, and L. Xie, "A diffusion-based framework for multi-class anomaly detection," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 38, no. 8, 2024, pp. 8472–8480.
- [62] X. Zhang, N. Li, J. Li, T. Dai, Y. Jiang, and S.-T. Xia, "Unsupervised surface anomaly detection with diffusion probabilistic model," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 6782–6791.
- [63] A. Bifet and R. Gavalda, "Learning from time-changing data with adaptive windowing," in *Proceedings of the 2007 SIAM international conference on data mining*. SIAM, 2007, pp. 443–448.
- [64] R. C. Cavalcante, L. L. Minku, and A. L. Oliveira, "Fedd: Feature extraction for explicit concept drift detection in time series," in *2016 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2016, pp. 740–747.
- [65] D. T. J. Huang, Y. S. Koh, G. Dobbie, and R. Pears, "Detecting volatility shift in data streams," in *2014 IEEE International Conference on Data Mining*. IEEE, 2014, pp. 863–868.
- [66] G. S. Na, D. Kim, and H. Yu, "Dilof: Effective and memory efficient local outlier detection in data streams," in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2018, pp. 1993–2002.
- [67] J. Nie, X. Wang, R. Hou, G. Li, H. Chen, and W. Zhu, "Dynamic spatio-temporal graph reasoning for videoqa with self-supervised event recognition," *IEEE Transactions on Image Processing*, vol. 33, pp. 4145–4158, 2024.



Jiaqi Zhu received the B.E. degree in automation from the Nanjing Institute of Technology, Nanjing, China, in 2019. She is currently a Ph.D. student of School of Automation at the Beijing Institute of Technology, Beijing, China. Her research interests include deep learning, anomaly detection, and time series analysis.



Shaofeng Cai received the B.Sc. degree in computer science from Peking University, China, and the Ph.D. degree in computer science from the National University of Singapore, in 2016 and 2021, respectively. He is currently a senior research fellow with the School of Computing, National University of Singapore. His research interests include adaptive neural architectures and techniques, in-database analytics, tabular deep learning, and anomaly detection.



Jie Chen (Fellow, IEEE) received the B.Sc., M.Sc., and Ph.D. degrees in Control Theory and Control Engineering from the Beijing Institute of Technology, Beijing, China, in 1986, 1996, and 2001, respectively. He was the President of Tongji University, Shanghai, China, during 2018–2023.

He is currently a Professor with the Harbin Institute of Technology, Harbin, China, and serves as the Director of the National Key Lab of Autonomous Intelligent Unmanned Systems at the Beijing Institute of Technology. His research interests include complex systems, multiagent systems, multiobjective optimization and decision,

constrained nonlinear control, and optimization methods. He is currently the Editor-in-Chief of Unmanned Systems and the Journal of Systems Science and Complexity. He has served on the editorial boards for several journals, including the IEEE Transactions on Cybernetics, International Journal of Robust and Nonlinear Control, and Science China Information Sciences.

He is a Fellow of IFAC, and a member of the Chinese Academy of Engineering.



Fang Deng (Fellow, IEEE) received the B.E. and Ph.D. degrees in control science and engineering from the Beijing Institute of Technology, Beijing, China, in 2004 and 2009, respectively.

He is currently a Professor at the School of Automation, Beijing Institute of Technology. His current research interests include machine learning, intelligent information processing, and control of renewable energy resources.



Beng Chin Ooi (Fellow, IEEE) received the B.Sc. and Ph.D. degrees from Monash University, Australia, in 1985 and 1989 respectively. He is the Lee Kong Chian Centennial professor and an NGS faculty member with the National University of Singapore. His research interests include database system architectures, performance issues, indexing techniques, and query processing, in the context of multimedia, spatio-temporal, distributed, parallel, blockchain, and in-memory systems. He served as the editor-in-chief of the IEEE Transactions on

Knowledge and Data Engineering (2009–2012), a trustee board member, and the president of the VLDB Endowment (2014–2017).

He is a fellow of ACM, CCF, Singapore National Academy of Science, and Singapore Academy of Engineering. He is a foreign member of Academia Europaea and Chinese Academy of Sciences.



Wenqiao Zhang received the Ph.D. degree from the Zhejiang University, Hangzhou, China, in 2021. He is an assistant professor at the College of Software, Zhejiang University, China. His current research interests include cross-media analysis and computer-aided healthcare. So far, he has authored more 50 papers in top-tier scientific journal/conference such as the IEEE Transactions on Visualization and Computer Graphics, AAAI, WWW, NeurIPS, CVPR, ACL, ACM-MM, KDD.